

数据库系统原理

课程知识汇总

声明

本资料仅供复习使用！

由于汇总知识的部分文字与代码问题，未经允许，不得抄袭，不得转发，不得打印，不得复印。若出现法律争议，本人概不负责！

张龠泮

2025 年 6 月

数据库系统原理 课程知识汇总

目录

1 数据库基础概念	1
1.1 数据库 4 个概念	1
1.1.1 数据(DATA)	1
1.1.2 数据库(DB)	1
1.1.3 数据库管理系统(DBMS)	1
1.1.4 数据库系统(DBS)	1
1.2 数据管理技术的发展阶段	2
1.3 数据模型	3
1.3.1 数据描述	3
1.3.2 概念数据模型	5
1.3.3 逻辑数据模型	5
1.3.3.1 层次模型	5
1.3.3.2 网状模型	6
1.3.3.3 关系模型	7
1.4 数据库系统结构	8
1.4.1 三级模式结构	8
1.4.1.1 外模式(external schema)	8
1.4.1.2 模式(schema)	8
1.4.1.3 内模式(internal schema)	8
1.4.2 两级映象功能	9
1.4.2.1 模式/内模式映象	9
1.4.2.2 外模式/模式映象	10
1.5 数据库系统(DBS)组成	10
2 关系数据库	12
2.1 关系数据结构及形式化定义	12
2.1.1 关系	12
2.1.2 关系模式	14
2.1.3 关系数据库	14
2.2 关系代数	14
2.2.1 关系代数的基本操作	14
2.2.2 运算详解	14
2.2.3 扩充的关系代数操作	18
3 SQL 语言	19
3.1 数据定义	19
3.1.1 基本数据类型	19
3.1.2 SQL 模式的创建和删除	20
3.1.3 基本表的创建、修改和撤销	20
3.1.4 索引的创建和撤销	22
3.2 数据查询	22

3.2.1 单表查询	22
3.2.2 多表查询（连接查询）	25
3.2.3 嵌套查询（子查询）	26
3.2.4 其他类型查询	28
3.3 数据更新	29
3.4 视图	30
3.4.1 视图特点	30
3.4.2 视图操作	30
4 数据库安全性	32
4.1 计算机安全性、数据库安全性	32
4.2 用户、角色	32
5 数据库安全性	33
5.1 关系的完整性	33
5.1.1 实体完整性	33
5.1.2 参照完整性	33
5.1.3 用户定义完整性	34
5.2 完整性约束命名	35
5.3 触发器	35
5.3.1 定义触发器	35
5.3.2 执行触发器	37
5.3.3 删除触发器	37
6 关系的规范化设计	38
6.1 函数依赖	38
6.2 范式	39
6.2.1 1NF	39
6.2.2 2NF	39
6.2.3 3NF	39
6.2.4 BCNF	40
6.2.5 多值依赖（非考点）	40
6.2.6 4NF（非考点）	40
7 数据库设计	41
7.1 数据库设计概述	41
7.2 概念结构设计	42
7.3 逻辑结构设计	43
7.4 物理结构设计与 DB 的运行	44
8 数据库编程	45
8.1 数据库设计概述	45
8.2 过程化 SQL	45
8.2.1 过程化 SQL 结构	45
8.2.2 游标	46
8.2.3 存储过程	47

8.3 JDBC 编程	48
9 关系数据库存储管理.....	49
9.1 数据组织	49
9.1.1 逻辑组织方式与物理组织方式.....	49
9.1.2 记录表示 ㉟.....	49
9.1.3 块的组织	50
9.1.4 关系表的组织	51
9.2 索引结构 ㉟.....	52
9.2.1 顺序表索引	52
9.2.2 辅助索引	54
9.2.3 B+树索引	54
9.2.4 哈希索引	55
9.2.5 位图索引	56
10 查询处理和查询优化.....	57
10.1 关系数据库系统的查询处理与优化.....	57
10.2 代数优化	57
10.2.1 关系代数表达式等价变换规则	57
10.2.2 查询树的启发式优化 ㉟.....	58
10.3 物理优化	61
10.3.1 基于启发式规则的存取路径选择优化.....	61
10.3.2 基于代价的优化	61
10.3.2.1 统计信息	61
10.3.2.2 代价估算 ㉟.....	62
11 数据库恢复技术.....	64
11.1 事务	64
11.2 数据库故障	64
11.3 数据库恢复技术	65
11.4 恢复策略	66
11.5 具有检查点的恢复技术 ㉟.....	66
11.6 数据库镜像	68
12 并发控制.....	69
12.1 多事务执行问题 ㉟.....	69
12.2 封锁 ㉟.....	70
12.3 活锁与死锁	72
12.4 并发调度的可串行性 ㉟.....	73
12.4.1 可串行化调度	73
12.4.2 冲突可串行化调度	73
12.5 两段锁协议（非重点）	74
12.6 封锁粒度（非重点）	74

注：带有“㉟”的为复习重点

1 数据库基础概念

1.1 数据库 4 个概念

1.1.1 数据(DATA)

定义：描述事物的符号称为“数据”。

数据是数据库中存储的基本对象。

1.1.2 数据库(DB)

数据库(DB)是长期存放在计算机内、有组织的、可共享的数据集合。数据库中的数据按一定的数据模型组织、描述和存储，且具有较小的冗余度、较高的数据独立性和易扩展性，并可为多个用户共享。

三个基本特点：永久存储、有组织、可共享

1.1.3 数据库管理系统(DBMS)

DBMS 是一个是位于操作系统之上的数据管理软件，是一个“系统软件”。
(eg: MySQL Server)

DBMS 是用于建立、使用和维护数据库的软件。

- ❖ 建立：如何定义、组织和存储数据
- ❖ 使用：查询、插入、删除和修改数据（**SQL 语句**）
- ❖ 维护：保证数据的安全性、完整性

DBMS 主要功能如下：

- ❖ 数据定义功能
- ❖ 数据组织、存储和管理
- ❖ 数据操纵功能（**增删改查**）
- ❖ 数据库的事务管理和运行管理（**安全性、完整性、并发使用、恢复**）
- ❖ 数据库的建立和维护功能

1.1.4 数据库系统(DBS)

采用了数据库技术的计算机系统，包括 **DB、DBMS**、开发工具（eg: MySQL Command Line Client）、**应用系统**、硬件、软件、用户和**数据库管理员(DBA)**。

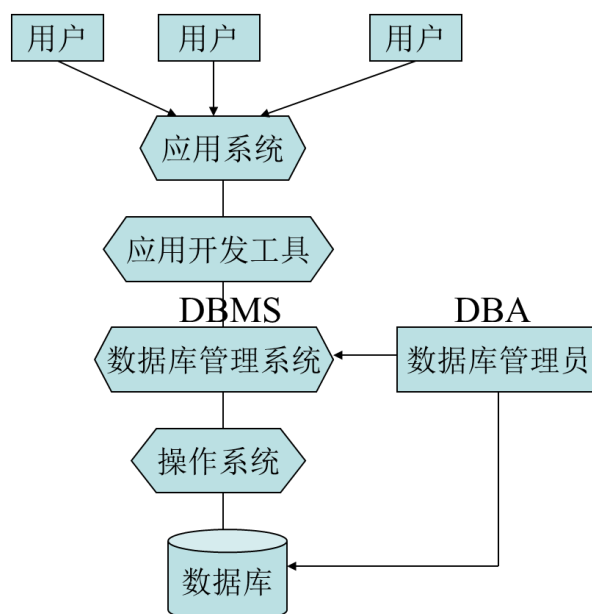


图 1 DBS

1.2 数据管理技术的发展阶段

❏ 人工管理阶段

外存：只有磁带、卡片和纸带 软件：只有汇编语言，无数据管理方面的软件

特点：数据不长期保存、应用程序管理数据、数据不具有独立性、数据不共享

❏ 文件系统阶段

外存：已有磁盘、磁鼓等直接存取存储设备

软件：出现高级语言和 OS，OS 中的文件系统是专门管理外存的数据管理软件

特点：数据可长期保存、由文件系统管理数据、数据共享性差，冗余度大、数据独立性差

❏ 数据库系统阶段

特点：①数据结构化②数据共享性高，冗余度低，易扩充③数据独立性高④数据由 DBMS 统一管理和控制（安全性、完整性、并发控制、恢复）

数据库系统与传统文件的根本区别：

- ❖ 文件系统：联机实时处理。记录内有结构，整体无结构。存取单位“记录”
- ❖ 数据库系统：分布处理。整体结构化，用数据模型描述。存取单位“数据项”

❏ 高级数据库技术阶段

分布式数据库、面向对象数据库、XML 数据库、图数据库

1.3 数据模型

模型：模型是现实世界特征的模拟和抽象

数据模型：一种模型，是现实世界数据特征的抽象，是表示实体类型及实体类型间联系的模型

数据模型应满足三个要求：

- ❖ 真实地模拟现实世界
- ❖ 容易理解
- ❖ 便于计算机上实现

1.3.1 数据描述

三个领域：

- ❖ 现实世界：存在于人脑之外的客观世界
- ❖ 信息世界：现实世界在人们头脑中的反映，以实体、实体集、属性、实体标识符等形式表示
- ❖ 机器世界：信息世界的信息在机器世界中的表示。以数据形式存储字段、记录、文件、关键码

信息世界的术语：

- ☆ 实体：可以相互区别的客观事物和概念的抽象
- ☆ 实体集：性质相同的同类实体的集合
- ☆ 属性：对实体某一特征的描述
- ☆ 实体标识符：能唯一标识每个实体的属性或属性集
- ☆ 实体型：用实体名及其属性名的集合来抽象和刻画同类实体，称为实体型。学生（学号，姓名）
- ☆ 域：属性的取值范围
- ☆ 联系：实体内部的联系和实体之间的联系
 - 实体内部的联系指的是实体各属性之间的联系
 - 实体之间的联系指的是不同实体集之间的联系

机器世界的术语:

- ☆ 记录: 字段的有序集合 (实体)
- ☆ 文件: 同一类记录或不同类记录集合 (实体集)
- ☆ 关键码(key): 能唯一标识文件中每条记录的字段或字段集 (实体标识符)
- ☆ 字段: 标记实体属性的命单位, 亦称数据项 (属性)

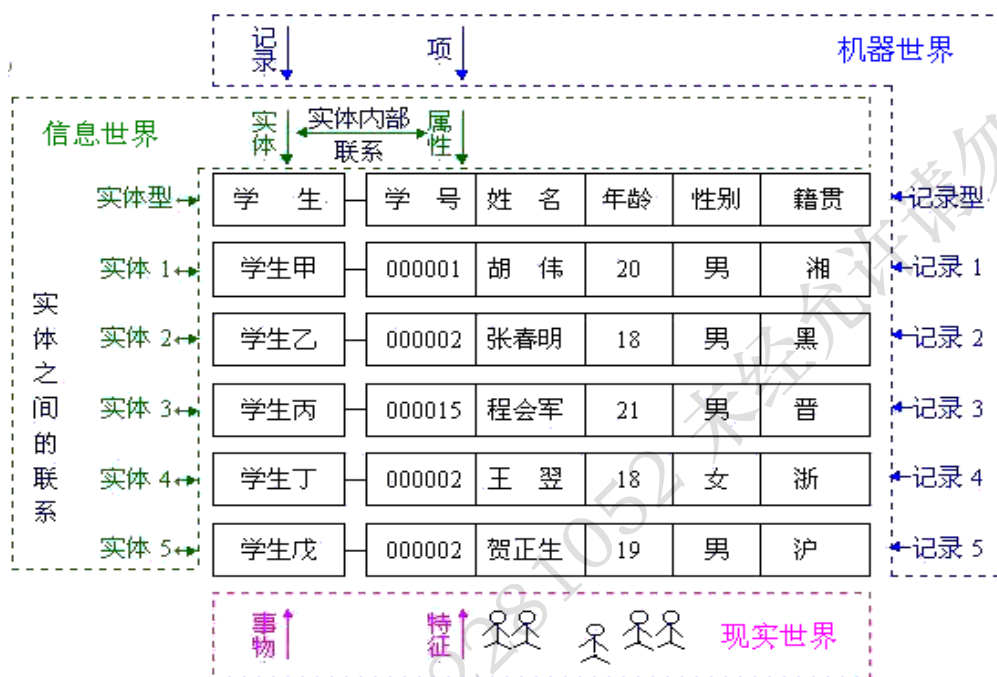


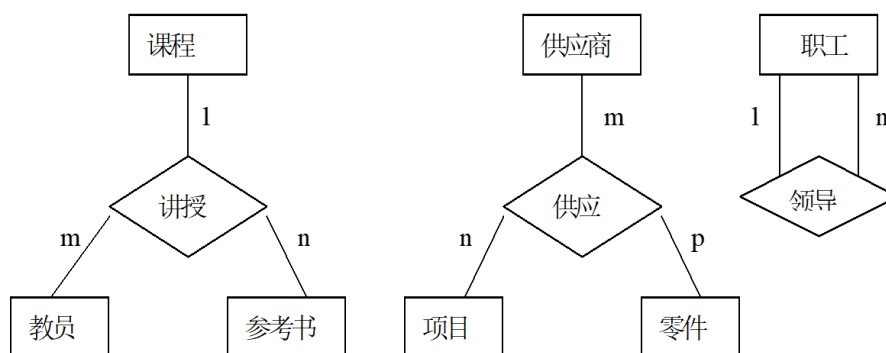
图2 三个世界的具体实例

实体集之间的联系:

1:1 (一对一联系)

1:n (一对多联系)

m:n (多对多联系)



一个实体型之间
一对多联系示例

图3 三个实体型之间的联系示例

1.3.2 概念数据模型

正确直观地反映客观事物及其联系,用于建立信息世界的数据库模型,是现实世界的第一层抽象(是现实到机器的中间层),强调语义表达功能。

PS: E-R 模型 (信息世界) 后面介绍, 面向对象数据模型不进行介绍

1.3.3 逻辑数据模型

这类模型涉及到计算机系统和数据库管理系统。包含三个组成部分:

(1)**数据结构**: 是所研究对象类型的集合。即对实体类型和实体间联系的表达和实现,是系统静态特征的描述。

(2)**数据操作**: 是指对数据库中各对象允许执行的操作的集合,包括检索和更新两类,是系统动态特征的描述。

(3)**数据完整性约束**: 给出数据及其联系应具有制约和依存规则,保证数据的正确,有效,相容。有实体完整性、参照完整性、用户定义完整性。

1.3.3.1 层次模型

树型结构组织数据,以记录类型为结点,以结点间联系为边的有序树/森林。

有且只有一个结点没有双亲结点,即根结点,根以外的其他结点有且只有一个双亲结点。

层次模型适合行政机构,家族关系等一对多关系的描述。如果要表示多对多的关系,必须分解。

插入操作: 如果没有相应的双亲结点,不能插入子结点

删除操作: 如果删除双亲结点,则相应的子女结点也被同时删除

修改操作: 应修改所有相应的记录,以保证数据的一致性

层次模型的存储结构有 2 种:

邻接法: 按层次树前序遍历顺序把所有记录依次邻接存放。

链接法: 用指针反映数据之间的层次联系。每个结点设两个指针,分别指向最左边的子女和最近的兄弟。

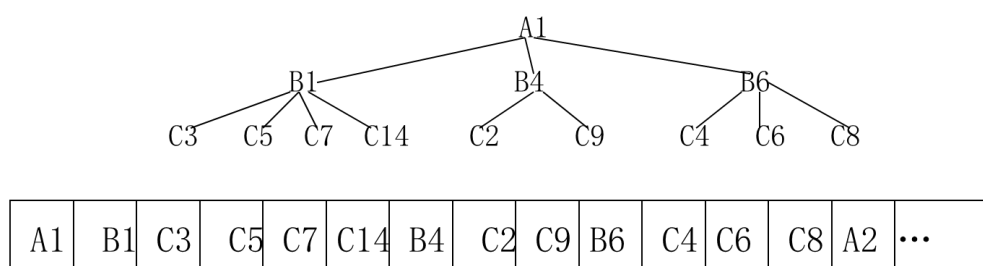


图4 邻接法示例，通过物理空间的位置相邻体现层次顺序

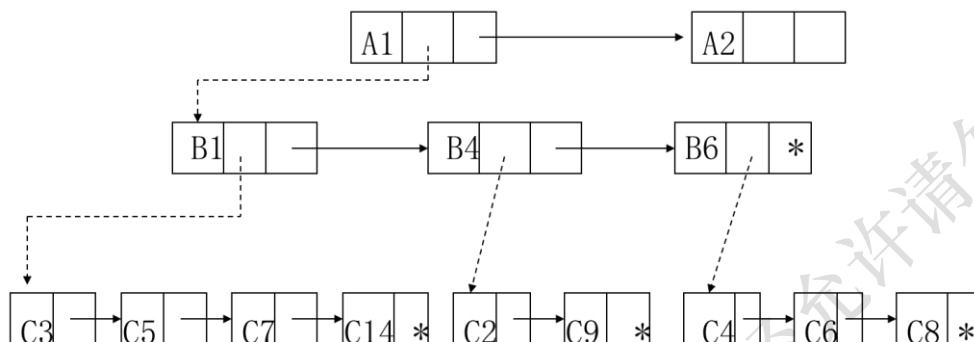


图5 链接法示例，虚线表示父子关系,实线表示兄弟关系

优点：记录之间联系通过**指针**实现，查询效率较高。提供良好的完整性支持。

缺点：

- ❖ 只能表示记录间的一对多联系，而多对多的联系会出现数据冗余。
- ❖ 结构严格、复杂，因此编程复杂
- ❖ 对插入和删除操作限制比较多
- ❖ 查询子女结点必须通过双亲结点

1.3.3.2 网状模型

用**有向图结构**表示实体及实体间联系的数据模型，它是以记录类型为结点，以结点间联系为边（适合于多对多关系）。常用的方法是链接法。

网状数据模型必须满足两个条件：

- ❖ 允许一个以上的结点无双亲结点
- ❖ 一个结点可以有多于一个双亲结点

操纵和完整性约束：

- ❖ 支持记录码的概念
- ❖ 保证联系中双亲记录和子女记录之间是一对多的联系
- ❖ 可以支持双亲记录与子女记录之间某些约束条件（必须有双亲才能插子女，删双亲连同删子女）

优点：在两个结点之间可以有**两种或多种联系**。记录之间的联系通过**指针**实现，M:N 联系也容易实现，**存取（查询）效率较高**。

缺点：结构复杂，编程复杂，不利于用户掌握。

1.3.3.3 关系模型

用**二维表**来表示实体集，用**外键**表示实体间的联系

术语：

- ❖ 关系：**一个关系对应一张表**
- ❖ 元组：表中一行即为一个元组
- ❖ 属性：表中一列即为一个属性
- ❖ **主码（关键码）**：表中某个属性组，可以唯一确定一个元组（**其他模型最大差别**）
- ❖ 域：属性的取值范围（**相同数据类型**）
- ❖ 关系模式：二维表的表头那一行

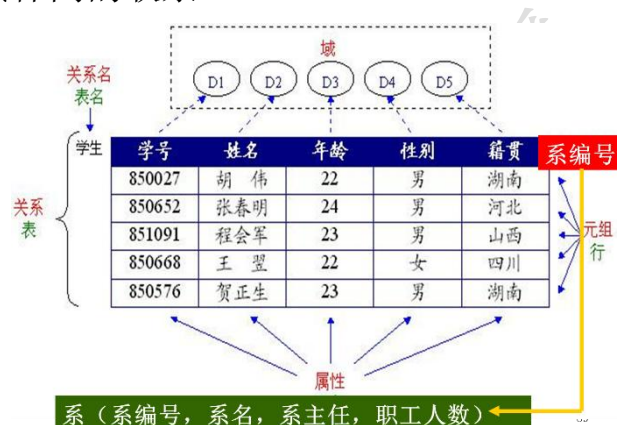


图6 关系模型示例

数据操纵：增删改查

关系的完整性约束条件：实体完整性、参照完整性、用户定义完整性

关系的存储结构：文件

关系的特点：

- ✧ （集合性质，不重复）一个关系（表）中不存在两个元组（行）在各个分量（或属性列）上完全相同。（不允许表中还有表）
- ✧ 行的次序无关
- ✧ 列的次序无关
- ✧ 每个分量必须是不可分的量

优点：

- ❖ 建立在严格的数学概念基础上
- ❖ 概念单一，实体/实体之间的联系都用关系表示，检索结果也是关系。结构简单清晰
- ❖ 关系模型存取路径对用户透明，更高的数据独立性、安全保密性。

缺点：查询效率不高

1.4 数据库系统结构

三级模式 两级映像

相关概念：

- ❖ 型(Type): 指对某一类数据的结构和属性的说明
- ❖ 值(Value): 型的一个具体赋值
- ❖ 模式(Schema): 数据库中全体数据的逻辑结构和特征的描述。模式反映的是数据的结构及其联系。
- ❖ 实例(Instance): 模式的一个具体值。反映的是数据库某一时刻的状态。同一模式有很多实例。

数据库系统的种类很多，但体系结构上有共同特征

1.4.1 三级模式结构

1.4.1.1 外模式(external schema)

又称“子模式”或“用户模式”，是模式的子集。最接近于用户的一级。

一个数据库可以有多个外模式。

每个用户必须使用一个外模式，一个外模式可以有数个不同的用户使用，一个应用程序只能使用一个外模式。用户的所有操作都是针对用户视图进行。

优点：①接口简单、使用方便 ②数据独立性增加 ③数据安全保密性增加

1.4.1.2 模式(schema)

是数据库中全部数据的逻辑结构和特征的描述。是所有用户的公共数据视图（全局的数据视图）。

一个数据库只存在一个 DBA 视图，即 一个数据库只有一个模式

描述概念记录类型、记录间的联系、操作、数据的完整性和安全性等

1.4.1.3 内模式(internal schema)

又称“存储模式”

是 数据库内部的组织方式。 一个数据库只有一个内模式。

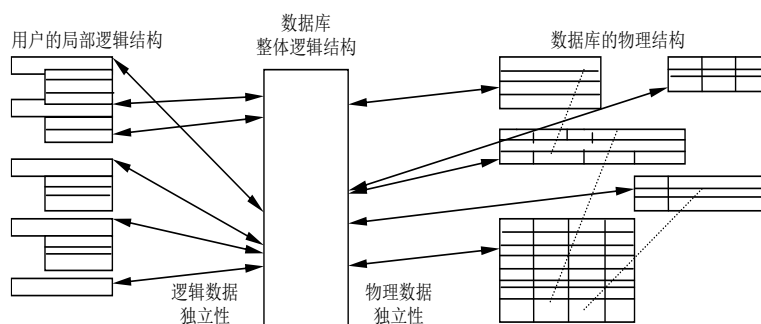


图7 三级模式，两级映像结构图示

1.4.2 两级映像功能

数据独立性：应用程序与数据间相互独立，不受影响。分为逻辑独立性、物理独立性

两级映像保证数据库系统的数据具有较高的逻辑独立性和物理独立性。

两级映像保证数据库外模式的稳定性，从而保证应用程序的稳定性。

两级映像简化了程序的编制，减少程序的维护和修改

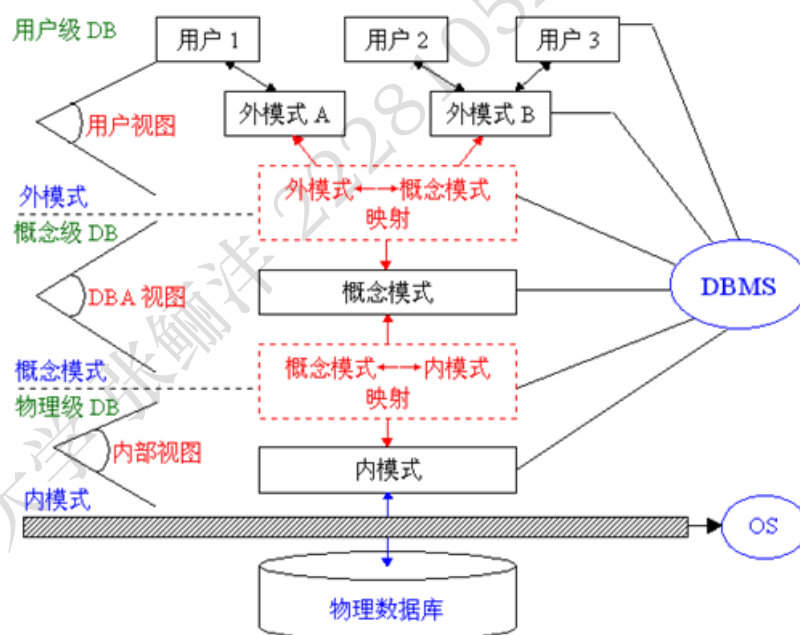


图7 三级模式，两级映像该概念图示

1.4.2.1 模式/内模式映像

用于定义模式和内模式间的对应性，定义了数据库全局逻辑结构与存储结构之间的对应关系，保证了数据与程序的物理独立性。

模式/内模式映像是唯一的（数据库只有一个模式，只有一个内模式）

模式/内模式映像一般是在模式中描述的。（修改时，内模式与映射变，模式不变）

物理独立性：用户的应用程序与 DB 中数据的**物理存储**是相互独立的。（用户程序不需了解物理结构，只需了解数据逻辑结构，减少用户程序修改）

1.4.2.2 外模式/模式映象

用于定义外模式和模式间的**对应关系**，保证了数据与程序的**逻辑独立性**。

每一个外模式，**数据库系统都有一个外模式/模式映象**

外模式/模式映象一般是在**外模式**中描述的。（修改时模式与映射变，**外模式不变**）

逻辑独立性：用户的应用程序与 DB 中数据的**逻辑结构**是相互独立的。（表结构，改变会影响外模式的变化，较大影响应用程序）

1.5 数据库系统(DBS)组成

1. 数据库

物理数据库：应用数据的集合，是 DB 的主体。

描述数据库：关于各级数据结构的描述。

2. 人员

数据库管理员（DBA）、系统分析员、数据库设计人员、应用程序员、最终用户。

DBA 职责：

- （1） 模式定义，决定数据库中信息内容和结构
- （2） 内模式定义，决定数据库的存储结构和存取策略
- （3） 根据要求修改数据库的模式和内模式
- （4） 对数据库访问权限的定义，保证数据安全性
- （5） 完整性约束条件的说明
- （6） 监控数据库的使用和运行（处理出现的问题）
- （7） 数据库的改进和重组重构（改进数据库设计）

3. 硬件

包括 CPU、内存、外存、I/O 设备、数据通道等设备

4. 软件

DBMS、OS、各种与数据库接口的高级语言及编译系统、应用开发工具、数据库应用系统。

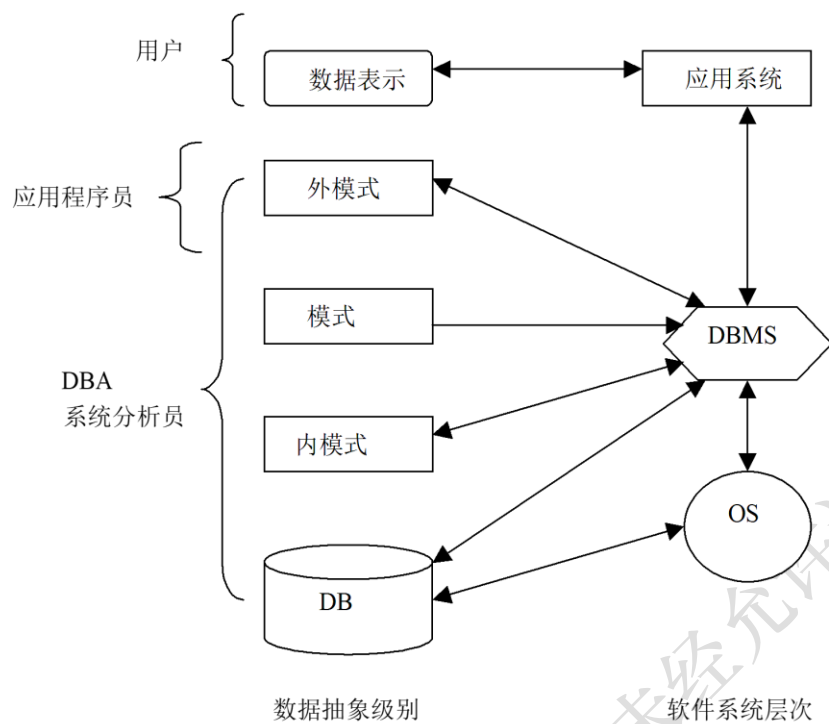


图 8 各种人员数据视图

2 关系数据库

关系数据模型：用二维表来表示实体集，用外键表示实体间的联系

数据模型三要素：

- ❖ **数据结构**：全部数据及相互联系都被组织成关系
- ❖ **数据操作**：查询和更新
- ❖ **完整性规则**：实体完整性、参照完整性、用户定义完整性
- ❖

2.1 关系数据结构及形式化定义

2.1.1 关系

单一的数据结构——关系

现实世界的实体以及实体间的各种联系均用关系来表示

逻辑结构——二维表

建立在集合代数的基础上

域 (Domain)：一组具有相同数据类型的值的集合

笛卡尔积 (Cartesian Product)

给定一组域 $D_1, D_2, \dots, D_n$ ，这些域中可以有相同 $D_1, D_2, \dots, D_n$ 的笛卡尔积为：

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, \&i = 1, 2, \dots, n\}$$

关系 (Relation)

$D_1 \times D_2 \times \dots \times D_n$ 的子集叫作在域 $D_1, D_2, \dots, D_n$ 上的关系，表示为

$$R(D_1, D_2, \dots, D_n)$$

R ：关系名

n ：关系的目或度 (Degree)

关系中的每个元素是关系中的元组，通常用 t 表示。

单元关系 ($n=1$) 二元关系 ($n=2$)

关系的表示：关系是二维表，表的每行对应一个元组，表的每列对应一个域属性：“每列起一个名字”， n 目关系必有 n 个属性

键(码):

- ❏ 候选键(码): 关系中的某一属性组的值能唯一地标识一个元组, 而其子集不能, 则称该属性组为候选键
- ❏ 主键(主码): 多个候选键中的一个
- ❏ 外键(外码): 如果关系 R 的某一个或一组属性不是本身的码, 而是另一关系 S 的主码, 则称该属性或属性组是 R 的外键

❏ 主属性: 候选码的诸属性

❏ 非主属性: 不包含在任何候选码中的属性

❏ 全码: 关系模式的所有属性是该关系模式的候选码, 称为全码

注意: 主属性是候选码中的属性, 候选码由主属性构成, 但主属性不一定是

候选码

举例: 假设你有一个学生表 Student(SID, Name, Email, Phone), 其中有两个候选码:

- 候选码 1: SID
- 候选码 2: (Email, Phone)

那么:

- 候选码有两个: SID 和 Email + Phone。
- 主属性 是出现在候选码中的属性: SID, Email, Phone
- 非主属性 是没出现在任何候选码里的: Name

注意:

- SID 是一个候选码(整组), 同时它里面的属性 SID 是主属性。
- Email 是主属性, 但它自己不是候选码(因为 Email 不能唯一标识学生, 要配合 Phone 才行)。

关系的性质:

- 一个关系中不存在两个元组在各个分量(或属性)上完全相同
- 行的次序无关
- 列的次序无关
- (规范化) 关系每个分量必须是不可分的量(分量必须取原子值)
- 列的分量是同一类型的数据, 来自同一域

三类关系:

- ☆ 基本表: 实际存在的表, 是实际存储数据的逻辑表示
- ☆ 查询表: 查询结果对应的表
- ☆ 视图表: 由基本表或其他视图表导出的虚表, 不对应实际存储的数据

2.1.2 关系模式

“表头”：指出元组集合的结构+关系遵循的完整性约束

关系模式的定义包括：**模式名、属性名、值域名**和**完整性约束**。

关系模式仅仅是对数据特性的描述。

关系模式是**静态的**（型），**关系**是关系模式某一时刻的状态，是**动态的**（值）。

2.1.3 关系数据库

在一个给定的应用领域中，所有关系的集合构成一个关系数据库。

关系数据库模式是对整个关系数据库的描述（型），关系数据库是所有关系模式某一时刻对应的关系的集合（值）。

2.2 关系代数

关系数据库的数据操作分为查询和更新

关系代数语言：是用关系（集合）运算来表达查询要求的 DML 语言

关系代数的运算对象和结果都是关系。

运算符包括四类：集合运算符、专门关系运算符、算术比较符、逻辑运算符

2.2.1 关系代数的基本操作

关系代数是一组建立在关系上的高级运算，每个运算都以一个或多个关系作为它的运算对象，并且生成一个关系作为运算结果。

关系运算分两类：

❖ 传统的集合运算：

- 合并 $\cup$ 、相交 $\cap$ 、求差-（相减）、笛卡尔积 $\times$
- （运算从关系的水平（行）的角度来进行）

❖ 专门的关系运算：

- 选择 δ 、投影 π 、连接 $\bowtie$ 、求商 $\div$
- （运算不仅涉及行而且涉及列）

2.2.2 运算详解 $\bowtie$

1. 合并 $\cup$

（类似于取并集，**去重**）两个关系 R_1 , R_2 （属性数相同，且相应属性取自同一个域）的合并，是由属于 R_1 或属于 R_2 （或属于两者）的所有元组 t （不计重复元组）组成的一个新的关系。

2. 交 $\cap$

两个关系 R_1, R_2 (属性数相同, 且相应属性取自同一个域) 的相交, 是由既属于 R_1 又属于 R_2 的所有元组 t 组成的一个新的关系。

3. 求差-

两个关系 R_1, R_2 (属性数相同, 且相应属性取自同一个域) 的求差, 是由属于 R_1 而不属于 R_2 的所有元组 t 组成的一个新的关系。

4. 笛卡尔积 $\times$

两个关系 R, S (设 R 为 k_1 元关系, M 个元组, S 为 k_2 元关系, N 个元组) 的广义笛卡尔积, 是一个 (K_1+K_2) 列的元组的集合, 是由属于 R 的任何一个元组 t^1 和属于 S 的元组 t^2 连接而成的新元组 t 所组成的一个新关系。

eg:

$$D = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \times \begin{bmatrix} a \\ b \end{bmatrix} \Rightarrow D = \begin{bmatrix} 1 & a \\ 1 & b \\ 2 & a \\ 2 & b \\ 3 & a \\ 3 & b \end{bmatrix}$$

$R \times S$ 结果中元组的个数 (即基数, 行数) = R 和 S 中行数之积
属性的个数 (即元数, 列数) = R 和 S 中列数之和

5. 选择 δ

从现有关系中选择满足一定条件元组组成新的关系。(从行的角度进行运算)

6. 投影 π

从现有关系中选取某些属性 (列), 组成新的关系。(从列的角度进行运算)

eg:

$$\pi_{A_3, A_1}(R) = \{t \mid t = \langle t_3, t_1 \rangle \wedge \langle t_1, t_2, t_3 \rangle \in R\}$$

初始:

A_1	A_2	A_3
a_1	b_1	c_1
a_1	b_2	c_1
a_2	b_2	c_2

结果:

A_3	A_1
c_1	a_1
c_2	a_2

7. 连接 $\bowtie$

也称为 θ 连接, 它是从两个关系的笛卡尔积中选取属性间满足一定条件的元组。

(1) 普通 θ 连接

$$R \bowtie_{A\theta B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B] \}$$

θ 可以是 $<$, $>$, $\leq$, $\geq$, $=$, $\neq$

F_连接: 当 F 为公式时称为 F_连接 ($R \bowtie_F S$), F 为: $F1 \wedge F2 \wedge \dots \wedge Fn$, 每个 Fi 都是形如 $i\theta j$ 的式子

θ 连接举例:

$R1 \bowtie_{2>2} R2$

A_1	A_2	A_3
a	4	e
b	2	f
c	5	g
d	1	h

A_1	A_2	A_3
b	2	e
c	3	f
b	5	h

结果如下:

$R_1.A_1$	$R_1.A_2$	$R_1.A_3$	$R_2.A_1$	$R_2.A_2$	$R_2.A_3$
a	4	e	b	2	e
a	4	e	c	3	f
c	5	g	b	2	e
c	5	g	c	3	f

(2) 等值连接

所有公式中 θ 均为 $=$ 号

(3) 自然连接

一种特殊的等值连接。要求两个关系中进行比较的分量是相同的属性组, 并在结果中去掉重复的属性列。(从行和列的角度进行运算)

自然连接举例:

A_1	A_2	A_3
a	1	b
b	2	c
c	4	e

A_1	A_2
e	2
a	1
b	3
c	4

$R \bowtie S$ 结果如下:

A_1	A_2	A_3
A	1	B
c	4	e

自然连接与等值连接的区别：

- ❏ **自然连接**两个关系要求**值相等的属性名必须相等**，等值连接中不要求。
- ❏ **自然连接**的结果**去掉重复的属性名**，等值连接结果中不去掉重复属性名。
- ❏ 若两个关系没有公共属性，则其自然连接就转化为笛卡尔积操作。

8. 求商÷

给定关系 $R(X, Y)$ 和 $S(Y, Z)$ ，其中 X, Y, Z 为属性组。 **R 中的 Y 与 S 中的 Y 可以有不同的属性名，但必须出自相同的值域。** R 与 S 的除运算得到一个新的关系 $P(X)$ ， P 是 R 中满足下列条件的元组在 X 属性列上的投影；元组在 X 上分量值 x 的**象集** Y_x 包含 S 在 Y 上投影的集合。（从行和列的角度进行运算）

象集：给定一个关系 $R(X, Y)$ ， X 和 Y 为属性组。定义，当 $t[X]=x$ 时， x 在 R 中的象集为 $Z_x = t[Z] | t \in R, t[X] = x$

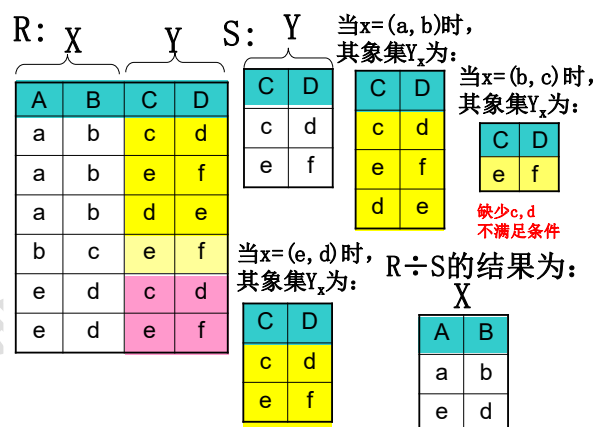
举例如下：

图9 求商案例 1

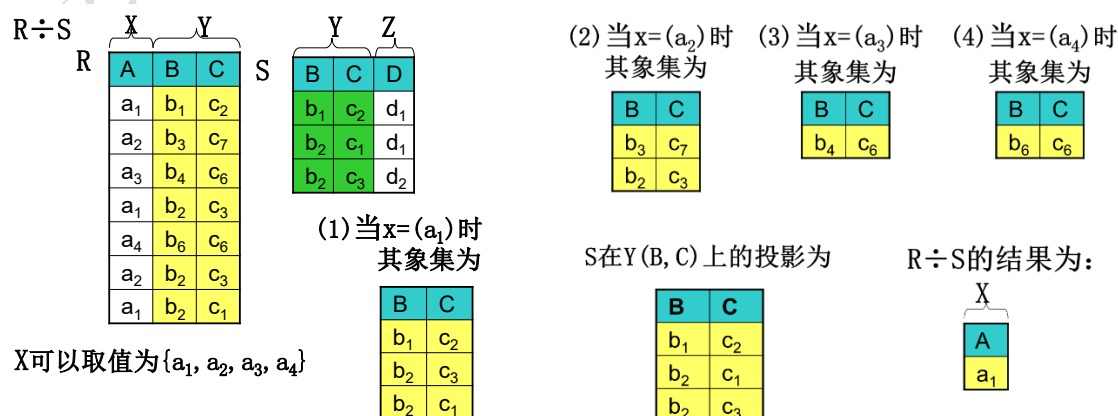


图10 求商案例 2

R 能被 S 除的充分必要条件：R 中包含 S 中的部分属性，(R 与 S 中的属性可以不同名，但必须有相同的值域) R 中有一些属性不出现在 S 中。

2.2.3 扩充的关系代数操作

1. 外联接 $\bowtie$

关系 R 和 S 做自然联接时，把 R 和 S 中原该舍弃的元组也保留在新关系中，同时在这些元组新增加的属性上填上空值 (Null)

R			S			R $\bowtie$ S			
A	B	C	B	C	D	A	B	C	D
a	b	c	b	c	d	a	b	c	d
b	b	f	b	c	e	a	b	c	e
c	a	d	a	d	b	c	a	d	b
			e	f	g				

R $\bowtie$ S			
A	B	C	D
a	b	c	d
a	b	c	e
c	a	d	b
b	b	f	null
null	e	f	g

图 11 外联接案例

2. 左外联接 $\ltimes$

关系 R 和 S 做自然联接时，只把 R 中原该舍弃的元组放到新关系中

3. 右外联接 $\rtimes$

关系 R 和 S 做自然联接时，只把 S 中原该舍弃的元组放到新关系中

“关系的完整性”见“5 数据库完整性”

3 SQL 语言

SQL 语言是一种关系数据库语言

SQL 语言不是一个应用程序开发语言

SQL 语言不是一个 DBMS，是 **DBMS** 为用户提供的交互语言。

SQL 语言支持关系数据库三级模式结构，但术语与传统关系模型术语不同。

表 1 SQL 语言与传统关系模型属于关联

关系模型中	SQL 中
模式	基本表
内模式	存储文件
外模式	视图 or 基本表

SQL 语言 4 分类：

❖ 数据定义语言（DDL）

CREATE SCHEMA / TABLE / VIEW / INDEX

（Schema 就像是数据库中的“文件夹”或“目录”，可以把不同用途的表归类存放，避免命名冲突，方便管理和授权。）

DROP SCHEMA / TABLE / VIEW / INDEX

ALTER TABLE （修改数据库中已存在表结构）

❖ 查询语言（QL）

SELECT...FROM...WHERE...

❖ 数据操纵语言（DML）

INSERT、UPDATE、DELETE

❖ 数据控制语言（DCL）

GRANT、REVOKE、COMMIT、ROLLBACK

3.1 数据定义

3.1.1 基本数据类型

数值型

❖ **Integer(int)**：长整数。

❖ **smallint**：短整数。

❖ **numeric (p, d)**：定点数，共 p 位（不包括小数点），右边 d 位。

❖ **real**：取决于机器精度的浮点数。

❖ **double precision**：取决于机器精度的双精度浮点数。

❖ **float(n)**：浮点数，精度至少为 n 位数字

字符型

- ❖ char(n)固定长度为 n 的字符串。
- ❖ varchar(n)最大长度为 n 的可变长字符串。

日期/时间型

- ❖ date: 日期（年、月、日），格式 YYYY-MM-DD。
- ❖ time: 时间（小时、分、秒），格式 HH:MM:SS。

3.1.2 SQL 模式的创建和删除

创建 SQL 模式即定义一命名空间，在这个空间中可以进一步定义该模式包含的数据库对象，例如基本表、视图、索引等。

创建模式：

CREATE SCHEMA <模式名> AUTHORIZATION <用户名>

如果没有指定<模式名>，<模式名> 隐含为<用户名>

eg: Create schema authorization wang

删除模式：

DROP SCHEMA <模式名> <CASCADE|RESTRICT>

eg: Drop schema wang cascade

CASCADE（级联）方式：删除模式同时把模式中所有数据库对象全部删除

RESTRICT（约束）方式：只有当模式中没有任何下属对象时才能执行

在 CREATE SCHEMA 中可以接受 CREATE TABLE, CREATE VIEW 和 GRANT 子句。

3.1.3 基本表的创建、修改和撤销

常用完整性约束：

- ❏ 主码约束：PRIMARY KEY
- ❏ 唯一性约束：UNIQUE
- ❏ 非空值约束：NOT NULL
- ❏ 参照完整性约束：FOREIGN KEY、REFERENCES
- ❏ 检查子句：CHECK

保留字不能用作表名、列名等

创建基本表：**CREATE TABLE** <基本表名>

(<列名 ><数据类型> [列级完整性约束条件]
 [,<列名 ><数据类型>[列级完整性约束条件]]...
 [,表级完整性约束条件]);

完整性约束条件被存入系统的数据字典中。

eg: 建立选课关系表 SC(SNO,CNO,G)

```
CREATE TABLE SC
(SNO CHAR(4),
CNO CHAR(4),
G SMALLINT,
PRIMARY KEY (SNO,CNO),
FOREIGN KEY(SNO)REFERENCES S(SNO),
FOREIGN KEY(CNO)REFERENCES C(CNO),
CHECK ((G IS NULL) OR (G BETWEEN 0 AND 100))
);
```

修改基本表结构：

❖ 对表增加列：不论原表中是否已存在数据，新增加的列一律为空值

ALTER TABLE <表名> **ADD** <列名> <数据类型> [完整性约束],...;

eg: ALTER TABLE S ADD ADDR CHAR(20);

❖ 对表增加新的完整性约束条件：

ALTER TABLE <表名> **ADD** <完整性约束>;

eg: ALTER TABLE C ADD UNIQUE (CN);

❖ 删除指定的完整性约束条件：

ALTER TABLE <表名> **DROP** <完整性约束名>;

SQL 不提供删除列的语法，Oracle 中允许删除列

❖ 修改原有列定义：

ALTER TABLE <表名> **ALTER COLUMN** <列名> <数据类型>;

eg: ALTER TABLE S ALTER COLUMN age int;

撤销基本表：

DROP TABLE <表名> [CASCADE|RESTRICT]

RESTRICT：如存在依赖该表的对象（视图、索引、触发器、存储过程、约束等），此表不能被删除。（默认）

CASCADE：删除该表的同时，相关的依赖对象被同时删除（表的数据、表上建立的索引和视图都自动被删除）。

3.1.4 索引的创建和撤销

建立索引是加快查询速度的有效手段

有些 DBMS 自动建立索引：PRIMARY KEY、UNIQUE

DBMS 自动完成维护索引，自动选择是否使用索引以及使用哪些索引

RDBMS 中索引一般采用 B+树、HASH 索引来实现（后续章节介绍）

索引的创建：

CREATE [UNIQUE] [CLUSTER] **INDEX** <索引名> **ON** <基本表名>

(<列名 1 >[ASC|DESC]

[,<列名 2 >[ASC|DESC]]...);

UNIQUE：要求列中的值在基本表中不重复。

CLUSTER：创建聚簇索引（经常更新的列不宜建立聚簇索引）

eg: **CREATE UNIQUE INDEX ST ON S(SNO);**

索引建立后由系统维护，不需用户干预。

索引的撤销：

DROP INDEX <索引名>

删除索引时，系统会同时从数据字典中删除有关该索引的描述。

3.2 数据查询

SELECT FROM WHERE

3.2.1 单表查询

选择表中若干列(SELECT)

查询指定列（或全部列）

select sno, sname, age, sex from s;

select * from s;

查询经过计算的列

select sno, sname, **2024-age as <新属性列名>** from S

🔍 选择表中若干元组(WHERE)

取消值重复的行

Select **distinct** sno from sc; (**Distinct**取消重复的值)

查询满足条件的元组——where

eg: select sname, sno from s where age>18 and sex='女'

表 2 查询条件

查询条件	谓词
比较	=、>、<、>=、<=、<>、!=、!>、!<
确定范围	BETWEEN AND 、NOT BETWEEN AND
确定集合	IN 、 NOT IN
字符匹配	LIKE 、 NOT LIKE
空值	IS NULL 、 IS NOT NULL
多重条件	AND 、 OR、 NOT

☆ **like**——模糊查找是否符合特定字符格式。

<列名> like/not like 字符串（列必须为字符串）;

通配符：“_”可代表任一单个字符，“%”可代表任意多个字符

eg: 查询学号第二位是1的所有学生姓名。

select sname from s where sno **like** ' _1%';

查询时需要包含“_”时，使用“\”，配合使用 ESCAPE

eg: select cno from c where cname **like** 'DB_DE' **escape** '\'

如果 LIKE 后面的匹配串中不含通配符，则可用=取代 LIKE，!= 或 <>

取代 NOT LIKE

☆ **null**——判断字段值是否为 NULL，不可以用“=”“!=”

eg: **SELECT * FROM student WHERE phone IS NOT NULL;**

☆ **between**——查找属性值在指定范围内的元组

<Attr> Between b and c （等价于 Attr>=b and Attr<=c）

<Attr> not Between b and c （等价于 Attr>c OR Attr<b）

☆ **in**——用来查找属性值属于指定集合的元组

<元素> **[not] in** <结果集> （等价于“=”“!=”）

eg: select sno from s where age not in (17, 18);

🔗 查询结果排序(Order by)

SELECT 列名 1, 列名 2, ... FROM 表名 **ORDER BY 列名 [ASC|DESC];**

ASC: 升序 DESC: 降序

对于排序列含空值, 若按升序排列, 含空值的元组将最后显示; 若按降序排列, 含空值的元组将最先显示

eg: SELECT * FROM student ORDER BY score DESC, name ASC;

🔗 使用集合函数 (5 个)

1) **count**

count ([distinct] <列名>)——统计一列中值的个数, 不计算空值

count ([distinct] *)——计算元组的个数, 不管列值是否为空

2) **sum**([distinct] <列名>)

——计算一列的总和 (此列必须是数值型)

3) **avg**([distinct] <列名>)

——计算一列的平均值 (此列必须是数值型)

4) **max**([distinct] <列名>)

——求一列值中的最大值

5) **min**([distinct] <列名>)

——求一列值中的最小值

Distinct: 表示计算时要取消指定列中的重复值

除 count(*)外, 都跳过空值而只处理非空值。

eg:

```
SELECT
    COUNT(*)    AS 总人数,
    SUM(score)   AS 总分,
    AVG(score)   AS 平均分,
    MAX(score)   AS 最高分,
    MIN(score)   AS 最低分
FROM student WHERE cno='c2';
```

🔗 对查询结果分组

GROUP BY 用于将 **WHERE** 查询结果按某列或多列分组

HAVING 是在分组后进行过滤的条件 (用于集合函数后的筛选)

eg: 按学生班级统计平均成绩, 筛选平均分大于 80 的班级

```
SELECT class, AVG(score) AS avg_score
FROM student
GROUP BY class
HAVING AVG(score) > 80;
```

name	class	score
Alice	1	85
Bob	1	95
Carol	2	70
David	2	75
Eva	3	95

3.2.2 多表查询（连接查询）

一般格式：

[<表名 1>.<列名 1> <比较运算符> [<表名 2>.<列名 2>

[<表名 1>.<列名 1> BETWEEN [<表名 2>.<列名 2> AND [<表名 2>.<列名 3>

表 3 学生选课数据库

表 a Student（学生表）

Sno	Sname	Ssex	Sbirthdate	Smajor
20180001	李勇	男	2000-3-8	信息安全
20180002	刘晨	女	1999-9-1	计算机科学与技术
20180003	王敏	女	2001-8-1	计算机科学与技术
20180004	张立	男	2000-1-8	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180006	赵明	男	2000-6-12	数据科学与大数据技术
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

表 b Course（课程表）

Cno	Cname	Ccredit	Cpno
81001	程序设计基础与 C 语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python 语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

表 c SC（选课表）

Sno	Cno	Grade	Semester	Teachingclass
20180001	81001	85	20192	81001-01
20180001	81002	96	20201	81002-01
20180001	81003	87	20202	81003-01
20180002	81001	80	20192	81001-02
20180002	81002	98	20201	81002-01
20180002	81003	71	20202	81003-02
20180003	81001	81	20192	81001-01
20180003	81002	76	20201	81002-02
20180004	81001	56	20192	81001-02
20180004	81002	97	20201	81002-02
20180005	81003	68	20202	81003-01

❏ 等值与非等值连接查询——连接运算符为“=”

eg: 找出平均成绩 90 以上的女生学号姓名

```
select sno, sname from s, sc where s.sno=sc.sno and sex= '女'
```

group by sno, sname (缺少 sname 会导致 SQL 不知道同一个 sno 下多个 sname 应该选哪个)
having avg(grade)>90 ;

```
SELECT age, sno, AVG(grade)
```

```
FROM s, sc
```

```
WHERE s.sno = sc.sno
```

```
GROUP BY age; (X) (SQL 不知道同一个 age 下多个 sno 应该选哪个)
```

如果使用了分组子句，则查询列表中的每个列要么是分组依据列（GROUP BY），要么是聚集函数

❏ 自身连接——同一个表的不同元组之间的连接必须给表取别名，当作两个不同的表来处理。

eg: 查询每一门课的间接先修课课序号（即先修课的先修课）

```
SELECT FIRST.Cno, SECOND.Cpno
```

```
FROM Course FIRST, Course SECOND
```

```
WHERE FIRST.Cpno = SECOND.Cno;
```

❏ 外连接

3.2.3 嵌套查询（子查询）

执行顺序由里往外，子查询的结果作为父查询的条件。

```
Select sname
```

```
from s
```

```
where sno in
```

```
(select sno from sc
```

```
where cno='C2');
```

可分成不相关子查询和相关子查询

❏ 不相关子查询（一般子查询）

不相关子查询一般使用谓词 IN

子查询的查询条件不依赖于父查询，子查询可以独立执行

执行过程：先执行子查询，子查询只执行一次，子查询的结果作为父查询的条件。根据子查询的结果再执行父查询。

特点:

- ☆ 能独立运行, 子查询条件不依赖父查询
- ☆ 只能运行一次
- ☆ 先执行子查询

eg: 找出平均成绩 90 以上的女生学号姓名 (之前出现)

```
select sname from s where sno in
(select sno from s,sc where s.sno=sc.sno and sex='女'
group by sno
having avg(grade)>90); (√) (不相关子查询)
```

❏ 相关子查询

子查询的查询条件依赖于外层父查询的某个属性值

相关子查询一般使用谓词 exists

特点:

- ☆ 子查询不能独立运行。
- ☆ 子查询多次运行
- ☆ 先执行外查询

eg: 查询有一门课程成绩等于 95 分的学生学号和姓名。

```
SELECT sno, sname
FROM s
WHERE EXISTS (
    SELECT *
    FROM sc
    WHERE s.sno = sc.sno
    AND grade = 95
);
```

外层表 s 开始扫描每一个学生 (逐行);
对于每个学生, 执行内部 SELECT * FROM sc;
判断是否存在一条记录 sc 满足:
s.sno = sc.sno (就是该学生)
grade = 95 (该学生有一门课成绩为 95)
如果 EXISTS 子查询返回非空结果 (存在匹配), 则该 s 行满足条件, 保留;
否则, 当前学生被过滤掉。

❏ 带有 IN 谓词的子查询

当子查询的结果是一个集合时, 经常使用带 IN 谓词的子查询。

<元素> in <子查询的结果集>

❏ 带有比较运算符的子查询

当用户能知道内层子查询返回的是单值时, 可以用比较运算符代替 IN

eg: 找出每个学生超过他选修课程平均成绩的课程号。

```
SELECT Sno, Cno FROM SC x
WHERE Grade >= (SELECT AVG(Grade) FROM SC y WHERE y.Sno=x.Sno);
```

带有 ANY 或 ALL 谓词的子查询

ANY: 子查询结果集的某个值

<表达式> <比较运算符> any <子查询结果集>

ALL: 子查询结果集的所有值

<表达式> <比较运算符> all <子查询结果集>

eg: not in ↔ <> all

in ↔ = any

max ↔ < any

min ↔ < all

带有 EXISTS 谓词的相关子查询

[not] exists (<子查询>)

带有 EXISTS 谓词（存在量词）的子查询**不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”**。若子查询结果为非空，则父查询的 **WHERE 子句返回真值**，否则，返回假值。

SQL 语言中没有提供全称量词，但是可以用“非”的方式转换 exists

eg: 检索选修全部课程的学生姓名（相当于查询这样的学生，没有一门课是他没选的）

```
SELECT sname
FROM s
WHERE NOT EXISTS (
    SELECT *
    FROM c
    WHERE NOT EXISTS (
        SELECT *
        FROM sc
        WHERE s.sno = sc.sno
        AND sc.cno = c.cno
    )
);
```

1. 遍历学生表 s			
2. 对于每个学生，遍历课程表 c: 如果发现某门课程 c.cno, 该学生 s.sno 在 sc 表中没有对应记录 → 学生没选该课			
3. 如果确实存在“没选的课” → NOT EXISTS(...) 成立 → 整体条件 NOT EXISTS(...) 不成立 → 学生被剔除			
4. 如果所有课程都能在 SC 表中查到对应记录 → NOT EXISTS(...) 不成立 → 外层 NOT EXISTS(...) 成立 → 保留该学生			
是否存在该学生没选的课程?	EXISTS	NOT EXISTS	NOT 学生被保留吗?
有	☑ 成立	✗ 不成立	✗ 不保留
没有	✗ 不成立	☑ 成立	☑ 保留

不相关子查询不一定能转化为多表的连接查询，而连接查询一定能用不相关子查询实现。

3.2.4 其他类型查询

集合查询:

- 集合操作主要包括 UNION（并）、INTERSECT（交）、MINUS（差）
- 参加集合操作的各个结果表的列数必须相同，对应项的数据类型也必须相同。

基于派生表的查询:

3.3 数据更新

☆ 插入数据

insert into <基本表名>[(列名表)] values (元组值);

eg: insert into s(sno,sname,age) values ('s1', '李涛', 19)

insert into s values ('s1','李涛',19,'男')

insert into <表名> [(<列名表>)] select 查询语句;

eg: insert into NoPass

select s.sno , sname , cno, grade

from s , sc

where s.sno =sc.sno and grade <60;

☆ 数据删除

DELETE FROM 基本表名 [WHERE 条件表达式];

DELETE 语句只能从一个基本表中删除满足 WHERE 子句中的条件的元组，即其后只能有一个基本表名。

DELETE 只删表中的数据，表的定义仍然在数据字典中。

☆ 数据修改

UPDATE 基本表名 SET 列名=值表达式 [WHERE 条件表达式];

update 语句只能修改一个基本表中满足 where 条件的元组的某些列值，即其后只能有一个基本表名。

eg: 将 c2 课程的非空成绩提高 10%。

UPDATE SC

SET GRADE=GRADE*(1+10%)

WHERE CNO='C2' AND GRADE IS NOT NULL

☆ 空值

属性定义中有以下**约束**的，不能取空值：

☆ NOT NULL

☆ UNIQUE

☆ PRIMARY KEY

空值运算：算术运算、比较运算、逻辑运算

3.4 视图

3.4.1 视图特点

- ✧ **虚表**，是从一个或几个基本表（或视图）导出的表
- ✧ 只存放视图的定义，**不存放视图对应的数据**
- ✧ 基表中的数据发生变化，从视图中查询出的数据也随之改变
- ✧ **不能修改视图**（TABLE、INDEX 可以 ALTER 修改）

视图作用：

简化用户的操作

使用户能以多种角度看待同一数据

对机密数据提供安全保护

适当的利用可以更清晰的表达查询

3.4.2 视图操作

创建视图

```
CREATE VIEW <视图名> [(<列名 1>,<列名 2>,...)]  
AS SELECT 查询语句 [WITH CHECK OPTION];
```

WITH CHECK OPTION 表示对视图进行 UPDATE,INSERT 和 DELETE 操作时要保证更新、插入和删除的行满足视图定义中的谓词条件

- ☆ **行列子集视图**：若一个视图从单个基本表导出，并且只是去掉基本表的某些行和某些列，但保留了主码。

eg: 建立一个男生行列子集视图，并保证进行插入和修改操作时该视图只有男生。

```
Create view M_student
```

```
as select sno,sname,age from s where sex='男'
```

```
with check option;
```

由于在定义视图时加了 with check option 子句，以后对该视图进行插入、修改和删除操作时，RDBMS 会自动加上 sex='男'的条件

- ☆ **带表达式视图**：在定义视图时根据需要设置一些派生列，这些派生列在基本表中并不实际存在，所以称为虚拟列。带虚拟列的视图称为带表达式视图。

eg: 建立一个带表达式的视图，反映学生出生年份的视图。

```
Create view bt_s(sno,sname,birth)
```

```
As select sno,sname, 2024-age from s;
```

☆ **分组视图**：带有集合函数和 GROUP BY 子句的查询来定义视图。

eg: 建立一个分组视图，反映学生的学号和平均成绩。

```
create view s_avg_g(sno,gavg)
as select sno,avg(g) from sc
group by sno;
```

❏ **撤销视图**

DROP VIEW <视图名>[CASCADE]

CASCADE: 将某个视图及其导出的所有视图全部删除

❏ **查询视图**

视图定义后，用户可以象对基本表一样对视图进行查询。

❏ **更新视图**

视图是从多个基本表联接操作导出的，**不允许更新**。

视图使用了分组和聚集操作，**不允许更新**。

视图为行列子集视图，则可以执行更新操作，更新操作自动转为原表上进行。

4 数据库安全性

只需理解“用户”“角色”及其关系即可，本章非重点。

4.1 计算机安全性、数据库安全性

安全性：保护计算机系统硬件、软件及数据

安全标准：TCSEC/TDI 标准、CC 标准

4.2 用户、角色

用户（User） 用户是数据库系统的认证主体

DB 最外层安全保护措施：用户标识、口令

存取控制机制组成：定义用户权限、合法权限检查（DBMS 安全子系统）

用户权限：GRANT 语句和 REVOKE 语句实现，定义存取权限称为授权

GRANT：授予权限

- GRANT <权限> [ON <对象类型> <对象名>] TO <用户> [WITH GRANT OPTION];

- WITH GRANT OPTION 可传播授权的权限给新用户

eg: GRANT ALL PRIVILIGES ON TABLE Student, Course TO U2, U3;

GRANT UPDATE(Sno), SELECT ON TABLE Student TO U4;

对属性列的授权时必须明确指出相应属性列名

REVOKE：回收权限

- REVOKE <权限> [ON <对象类型> <对象名>] FROM <用户> [CASCADE|RESTRICT];

eg: REVOKE SELECT ON TABLE SC FROM PUBLIC; （所有用户收回权限）

CREATE USER：创建数据库模式的权限

- CREATE USER <username> [WITH][DBA|RESOURCE|CONNECT]

角色（Role）是一组权限的命名集合，可以把角色看作是一个“职位”或“职责分组”

- 创建角色：CREATE ROLE <角色名>

- 授权角色：GRANT <权限> ON <对象类型> 对象名 TO <角色>

- 一个角色授予其他的角色或用户：GRANT <角色 1> TO <角色 2>

- 角色权限收回：REVOKE <权限> ON <对象类型> <对象名> FROM <角色>

用户是登录数据库的实体，角色是权限的集合。可以与 C++ 进行类比，角色是权限的模板（class），用户是权限的使用者（object）。

5 数据库安全性

数据失真因素：数据进入系统时的错误、系统故障、相互干扰、恶意破坏

5.1 关系的完整性

5.1.1 实体完整性

是关系模型必须满足的完整性约束条件，由关系系统自动支持。

基本关系 R 的主属性不能取空值。用 **PRIMARY KEY** 定义

单属性构成的码说明方法：

□ 列级约束条件：Sno CHAR(9) **PRIMARY KEY**

□ 表级约束条件：Sno CHAR(9), **PRIMARY KEY (Sno)**

多属性构成的码说明方法：

□ 表级约束条件：Sno CHAR(9), Cno CHAR(4), **PRIMARY KEY (Sno, Cno)**

插入或对主码列进行更新操作时，RDBMS 按照实体完整性规则自动检查：

1. 检查主码值是否唯一，如果不唯一则拒绝插入或修改
2. 检查主码的各个属性是否为空，只要有一个为空就拒绝插入或修改

5.1.2 参照完整性

是关系模型必须满足的完整性约束条件，由关系系统自动支持。

定义主--外码之间的引用规则。不引用不存在的实体，即确保外键所引用的主键在被引用的表中是存在的，在表级定义参照完整性

参照完整性，除了应该定义外码，还应定义外码列是否允许空值

FOREIGN KEY：定义哪些列为外码

REFERENCES：指明这些外码参照哪些表的主码

eg: **CREATE TABLE SC**

(Sno CHAR(9),

Cno CHAR(4),

PRIMARY KEY (Sno, Cno),

FOREIGN KEY (Sno) REFERENCES Student(Sno),

FOREIGN KEY (Cno) REFERENCES Course(Cno)

);

注意：

- ❖ 外键和相应的主键可以不同名，只要定义在相同的值域上即可

eg1: 职工（职工号，职工名，年龄，**部门编号**）

部门（**部门号**，部门名，负责人）

- ❖ 参照与被参照关系可以是同一个关系，表示了属性之间的联系。（关系内部的引用）如：

eg1:职工（**职工号**，姓名，年龄，部门号，**负责人职工号**）

也可以是两个不同的关系，表示实体之间的引用（关系间的引用）

- ❖ 外键值是否允许空，应视具体问题而定。（不可空 eg1，可空 eg2）

表 4 可能破坏参照完整性的情况及违约处理

被参照表 (例如 Student)	参照表 (例如 SC)	违约处理
可能破坏参照完整性 ←	插入元组	拒绝 (NO ACTION)
可能破坏参照完整性 ←	修改外码值	拒绝 (NO ACTION)
删除元组 →	可能破坏参照完整性	拒绝/级连删除(CASCADE)/设置为空值(SET-NUL)
修改主码值 →	可能破坏参照完整性	拒绝/级连修改(CASCADE)/设置为空值(SET-NUL)

5.1.3 用户定义完整性

是应用领域需要遵循的约束条件。关系模型提供定义和检验这类完整性的机制，不必用程序完成

属性上的约束条件——CREATE TABLE 时定义

□ 列值非空 (NOT NULL): Sno CHAR(9) **NOT NULL**,

□ 列值唯一 (UNIQUE): Dname CHAR(9) **UNIQUE**

□ 检查列值是否满足一个布尔表达式 (CHECK):

Ssex CHAR(2) **CHECK (Ssex IN ('男', '女'))**

如果不满足则操作被拒绝执行

元组上的约束条件——CREATE TABLE 时用 **CHECK** 定义

元组级的限制可以设置不同属性之间的取值的相互约束条件

eg1: CREATE TABLE Student

(Sname CHAR(8) NOT NULL,

Ssex CHAR(2),

CHECK (Ssex='女' OR Sname NOT LIKE 'Ms.%'));

如果不满足则操作被拒绝执行

5.2 完整性约束命名

CONSTRAINT 约束定义:

CONSTRAINT <完整性约束条件名>[PRIMARY KEY|FOREIGN KEY|CHECK]

eg: 建立学生登记表 Student, 要求学号是以 s 打头的 5 位字符, 后 4 位只能是数字, 姓名不能取空值, 年龄小于 30, 性别只能是“男”或“女”。

```
CREATE TABLE Student
```

```
(Sno CHAR(5)
```

```
CONSTRAINT C1 CHECK (Sno Like 's[0-9][0-9][0-9][0-9]'),
```

```
Sname CHAR(20)
```

```
CONSTRAINT C2 NOT NULL,
```

```
Sage NUMERIC(3)
```

```
CONSTRAINT C3 CHECK (Sage < 30),
```

```
Ssex CHAR(2)
```

```
CONSTRAINT C4 CHECK (Ssex IN ('男', '女')),
```

```
CONSTRAINT StudentKey PRIMARY KEY (Sno));
```

在 Student 表上建立了 5 个约束条件, 包括主码约束 (命名为 StudentKey) 以及 C1、C2、C3、C4 四个列级约束。

修改表中的完整性限制: 使用 **ALTER TABLE** 语句修改

可以先删除原来的约束条件, 再增加新的约束条件

eg: ALTER TABLE Student

```
DROP CONSTRAINT C1;
```

```
ALTER TABLE Student
```

```
ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000 AND 999999),
```

5.3 触发器

触发器 (trigger) 是用户定义在关系表上的一类由事件驱动的特殊过程

5.3.1 定义触发器

触发器: 事件-条件-动作规则, 当特定的系统事件发生时, 对规则的条件进行检查, 如果条件成立则执行规则中的动作, 否则不执行该动作

CREATE TRIGGER <触发器名>

{BEFORE|AFTER} <触发事件> ON <表名>

REFERENCING NEW|OLD AS<变量>

FOR EACH {ROW|STATEMENT}

[WHEN <触发条件>] <触发动作体>

表的**拥有者**才可以在表上创建触发器

☆ 触发器名:

可以包含模式名,也可以不包含模式名

同一模式触发器名必须是唯一的,触发器名和表名必须在同一模式下

☆ 表名:

触发器只能定义在基本表上,不能定义在视图上

基本表的数据发生变化时,将激活定义在该表上相应触发事件触发器

☆ 触发事件:

触发事件可以是 INSERT、DELETE、UPDATE 或组合

可以 **UPDATE OF <触发列>**,即进一步指明修改哪些列时激活触发器

AFTER/BEFORE 是触发的时机

□ AFTER 表示在触发事件的操作执行之后激活触发器

□ BEFORE 表示在触发事件的操作执行之前激活触发器

☆ 触发器类型:

行级触发器 (FOR EACH ROW): UPDATE 语句影响多少行就触发多少次

语句级触发器 (FOR EACH STATEMENT): 执行完 UPDATE 语句后触发动作体将执行一次 (默认)

☆ 触发条件:

触发器被激活时,只有当触发条件为真时触发动作体才执行,否则不执行

如果省略 WHEN 触发条件,则触发动作体在触发器激活后立即执行

☆ 触发动作体:

触发动作体可以是匿名 SQL 过程块,也可以是对已创建存储过程的调用

对于行级触发器,用户可以在过程体中使用 **NEW** 和 **OLD** 引用 **UPDATE/INSERT** 事件之后(前)的新(旧)值

如果触发动作体执行失败,激活触发器的事件就会终止执行,触发器的目标表或触发器可能影响的其他对象不发生任何变化

eg1: 将每次对表 Student 的插入操作所增加的学生个数记录到表 StudentInsertLog(numbers INT)中,需要提前创建此表

```
CREATE TRIGGER Student_Count
```

```
AFTER INSERT ON Student /*指明触发器激活的时间是在执行 INSERT 后*/
```

```
REFERENCING NEW TABLE AS Delta
```

```
FOR EACH STATEMENT /*语句级触发器,执行完 INSERT 后下面的触发动作体才执行一次*/
```

```
BEGIN
```

```
INSERT INTO StudentInsertLog (Numbers) SELECT COUNT(*) FROM Delta
```

```
END;
```

eg2: 定义一个 BEFORE 行级触发器，为教师表 Teacher 定义完整性规则：教授的工资不得低于 4000 元，如果低于 4000 元，自动改为 4000 元。

```
CREATE TRIGGER Update_Sal
BEFORE UPDATE ON Teacher /*对教师表插入或更新前激活触发器*/
REFERENCING NEW AS newTuple
FOR EACH ROW /*这是行级触发器*/
BEGIN /*定义触发动作体，这是一个 PL/SQL 过程块*/
    IF (newTuple.job='教授') AND (newTuple.sal < 4000) /*行级触发器可在过程体中使用插入或更新操作后的新值*/
    THEN newTuple.sal :=4000;
    END IF;
END;
```

5.3.2 执行触发器

指由触发事件激活，并由数据库服务器自动执行一个数据表上可能定义了多个触发器，遵循执行顺序：

执行 BEFORE 触发器→激活触发器的 SQL 语句→执行 AFTER 触发器

5.3.3 删除触发器

DROP TRIGGER <触发器名> ON <表名>;

触发器必须是一个已经创建的触发器，只能由具有相应权限的用户删除

6 关系的规范化设计

泛关系模式存在的问题：数据冗余、更新异常、插入异常、删除异常

产生问题原因：属性间约束关系（即数据间的依赖关系）太强

解决方案：分解关系

规范化目的：尽量消除插入、删除异常，修改复杂，数据冗余

规范化基本思想：逐步消除数据依赖中不合适的部分

6.1 函数依赖

数据依赖：通过一个关系中属性间值的相等与否体现出来的数据间的相互关系，分函数依赖和多值依赖。

函数依赖：反映属性或属性组之间相互依存，互相制约的关系，即反映现实世界的约束关系。只能根据语义来确定函数依赖性的存在与否。

概念：在一个关系中，属性组 X 的取值能唯一决定另一个属性组 Y 的取值，我们称 Y 函数依赖于 X ，记作 $X \rightarrow Y$ 。

属性间的联系决定函数依赖关系：设 X 、 Y 均是 U 的子集

□ X 和 Y 间联系是 1:1，则 $X \rightarrow Y, Y \rightarrow X$ 。

□ X 和 Y 间联系是 $M:1 (M > 1)$ ，则 $X \rightarrow Y$ 。

□ X 和 Y 间联系是 $M:N (M, N > 1)$ ，则 X 、 Y 间不存在函数依赖。

自反性：若 $Y \subseteq X \subseteq U$ ，则 $X \rightarrow Y$ 在 R 上成立

增广性：若 $X \rightarrow Y$ 在 R 上成立，且 $Z \subseteq U$ ，则 $XZ \rightarrow YZ$ 在 R 上成立

传递性：若 $X \rightarrow Y$ 和 $Y \rightarrow Z$ 在 R 上成立，则 $X \rightarrow Z$ 在 R 上成立

完全函数依赖、部分函数依赖、平凡函数依赖、非平凡函数依赖

完全依赖 ($X \xrightarrow{F} Y$)： X 所有属性都必须用上

部分依赖 ($X \xrightarrow{P} Y$)： X 中部分属性就能决定 Y ($\rightarrow$ 存在冗余)

平凡函数依赖： Y 包含于 X

非平凡函数依赖： Y 不包含于 X

6.2 范式

6.2.1 1NF

关系模式的所有域为简单域，其元素不可再分，即属性不能再分。

不满足 1NF 的关系称为非规范化关系(eg:工资(工号,姓名,工资(基本工资,煤电补贴)))

关系数据库至少满足 1NF

6.2.2 2NF

关系模式 R 及函数依赖集 F，R 的任何一个非主属性都完全函数依赖于码

eg: S-L-C(Sno, Sdept, Sloc, Cno, Grade) $\notin$ 2NF

S-C (Sno, Cno, Grade) $\in$ 2NF

S-L (Sno, Sdept, Sloc) $\in$ 2NF

2NF 存在传递函数依赖问题: $X \rightarrow Y, Y \rightarrow Z$, 并且 $Y \nrightarrow X$, 则 $X \rightarrow Z$ 即为 Z 传递函数依赖于 X, 可能导致插入和删除异常

6.2.3 3NF

关系模式 R 及函数依赖集 F，R 的任何一个非主属性都不传递函数依赖于它的任何一个候选码

eg: P={ SNO, SNAME, CITY, province }

F={ SNO $\rightarrow$ SNAME, SNO $\rightarrow$ CITY, CITY $\rightarrow$ province }

分解为 3NF:

SP1={ SNO, SNAME, CITY }

SP2={ CITY, province }

R={ SNO, TNO, CNO }

F={ TNO $\rightarrow$ CNO, (SNO, TNO) $\rightarrow$ CNO, (SNO, CNO) $\rightarrow$ TNO // 一名教师只能上一门课; 一门课可由多个教师上



候选码为 (SNO, TNO) 和 (SNO, CNO)

R 中没有非主属性, 故 R 为 3NF。

存在问题: 例如要插入某个教师能讲授某门课的信息, 而又没有学生选修该课时, 会插入异常。

分解 (BCNF): SC(SNO, CNO), TC(TNO, CNO)

3NF 存在数据插入异常风险

6.2.4 BCNF

F 中每个非平凡函数依赖 $X \rightarrow Y$ 的左部(决定因素)X 中必含有候选键(码)

$R \in 1NF$, 若 $X \rightarrow Y$ 且 $Y \not\subseteq X$, X 中必含有候选键(码), 则 $R \in BCNF$

eg: C(CNO, CNAME, TNAME) T(TNAME, SALARY)

- ☐ 非主属性对码完全函数依赖
- ☐ 非主属性不传递函数依赖于任何一个码
- ☐ 主属性对不含它的码完全函数依赖
- ☐ 主属性不传递函数依赖于任何一个码
- ☐ 没有属性完全函数依赖于一组非主属性

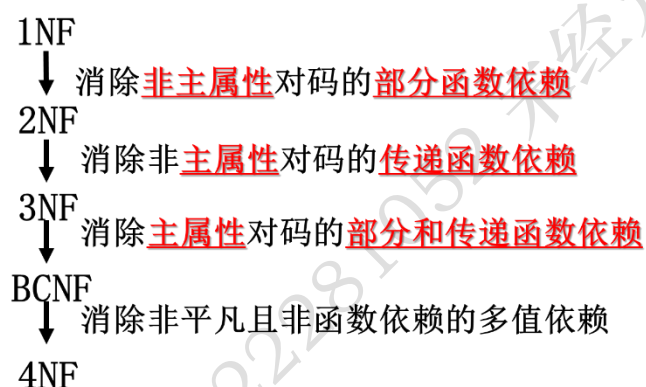


图 12 规范化基本步骤

6.2.5 多值依赖 (非考点)

性质: 对称性、传递性。

函数依赖可以看作是多值依赖的特殊情况, 即若 $X \rightarrow Y$, 则 $X \twoheadrightarrow Y$ 。

平凡的多值依赖/非平凡的多值依赖

6.2.6 4NF (非考点)

设 R 是一个关系模式, D 是 R 上的多值依赖集合。如果 D 中成立非平凡多值依赖 $X \twoheadrightarrow Y$ 时, X 必包含 R 的码, 那么称 R 是第四范式的模式。

7 数据库设计

数据失真因素：数据进入系统时的错误、系统故障、相互干扰、恶意破坏

7.1 数据库设计概述

数据库系统生存期：

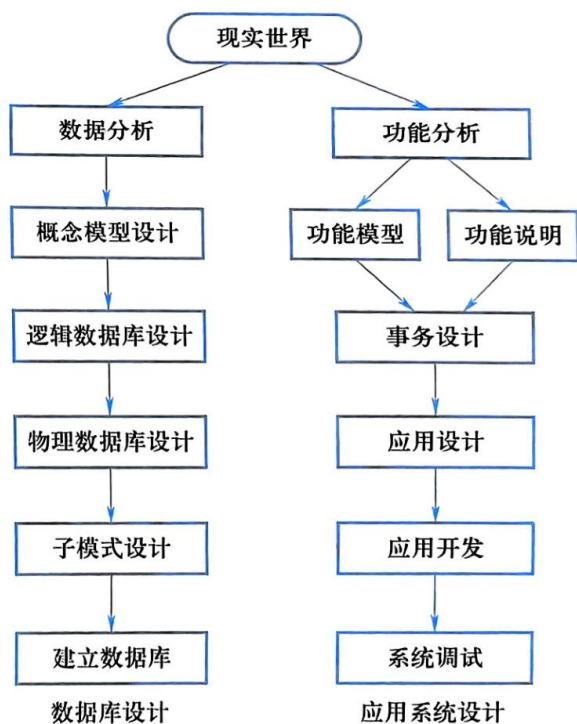


图 13 数据库设计和应用系统设计分离的设计

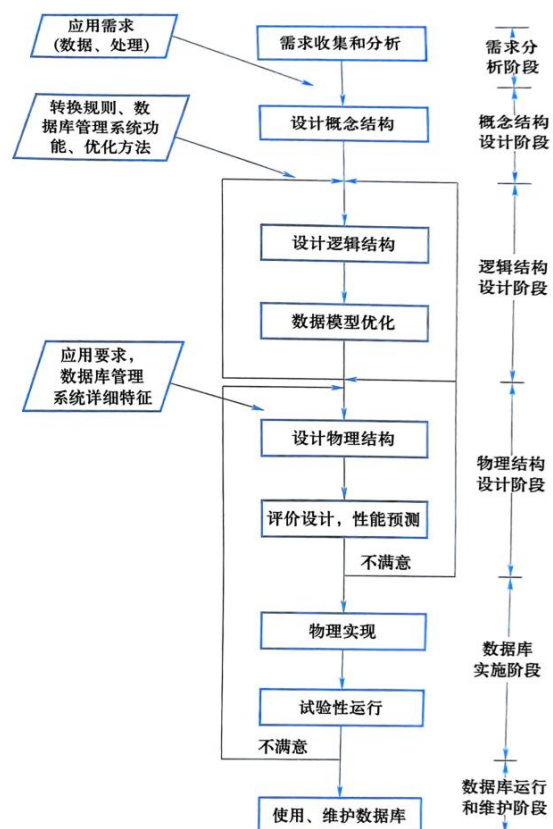


图 14 数据库设计的基本步骤

需求分析：

1. 调查需要处理的对象（部门等）
2. 了解原系统特征
3. 明确用户各种需求
4. 确定新系统功能
5. 考虑后续的扩充改变

需求分析目标：

分析用户活动，产生业务流程图

确定系统范围，产生系统范围图

分析用户活动涉及的数据，产生数据流图

分析系统数据，产生数据字典

数据字典：系统中各类数据描述的集合，是进行详细的数据收集和数据分析所获得的主要成果。应包括对以下几部分数据的描述：

1. **数据项：**不可再分的数据单位

数据项描述 = { 数据项名, 数据项含义说明, 别名, 数据类型, 长度, 取值范围, 取值含义, 与其他数据项的逻辑关系, 数据项之间的联系 }

2. **数据结构：**反映了数据之间的组合关系。

数据结构描述 = { 数据结构名, 含义说明, 组成: { 数据项或数据结构 } }

3. **数据流：**是数据结构在系统内传输的路径。

4. **数据存储：**数据结构停留或保存的地方，也是数据流的来源和去向之一。

5. **处理过程：**处理过程说明性信息的描述

7.2 概念结构设计

设计概念结构通常有自顶向下、自底向上、逐步扩张、混合策略 4 类方法。

常用策略：自顶向下地进行需求分析，自底向上地设计概念结构

概念结构的设计步骤：

1. 抽象数据并设计局部视图

2. 集成局部视图，得到全局的概念结构

概念模型：**实体联系模型（ER 模型）**

ER 图提供了表示实体，属性，联系的方法

ER 模型中的三要素：

实体：表示客观事物。

属性：表示客观事物的特征（属性）

联系：客观事物之间的联系

ER 图表示方法：

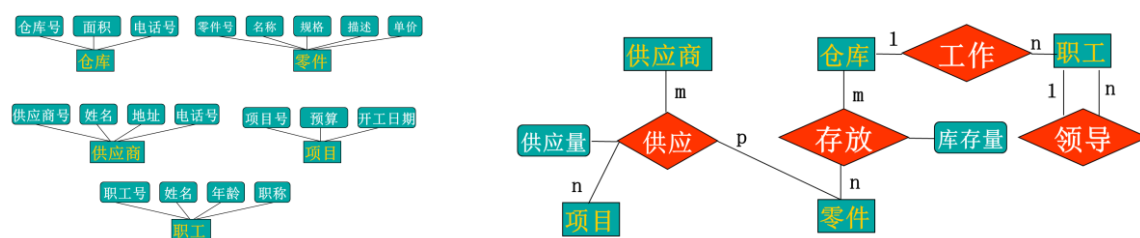
用矩形表示实体、用椭圆表示属性、用菱形表示实体间的联系，属性和实体间、实体和联系间用线段连接（同时在线段边上标上联系的类型）

建立实体-联系模型：

☆ 确定实体以及实体的属性

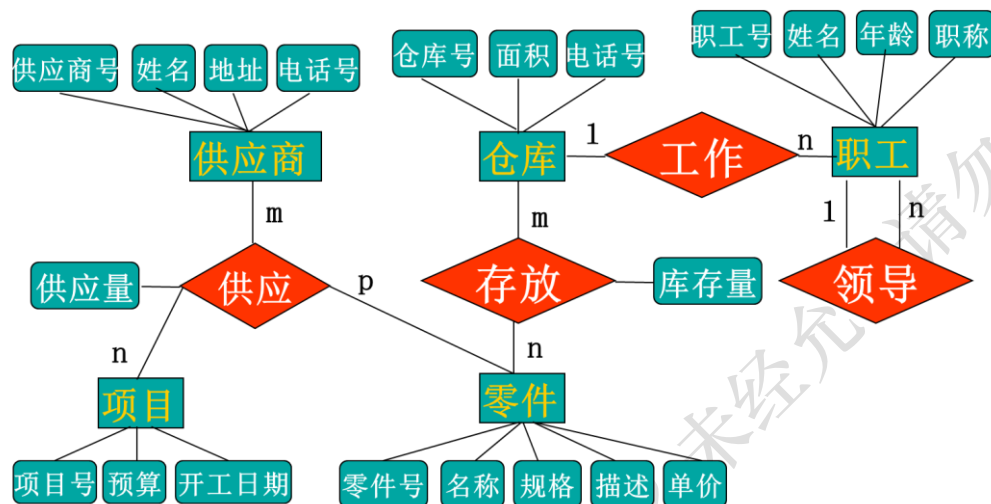
☆ 确定实体之间的联系以及联系的属性

☆ 将实体的属性都加上——>完成了完整的 ER 图



图(a) 确定实体及属性

图(b) 确定联系及属性



图(c) 综合汇总

图 15 ER 图建立过程

视图的集成：两种方式

多个分 E-R 图一次集成。

逐步集成，用累加的方式一次集成两个分 E-R 图。

集成局部 E-R 图步骤：

合并：解决各分 E-R 图之间的冲突，将分 E-R 图合并起来生成初步 E-R 图。

修改和重构：消除不必要的冗余，生成基本 E-R 图

7.3 逻辑结构设计

逻辑结构设计任务：把概念结构设计阶段设计好的基本 E-R 图转换为与选用的 DBMS 产品所支持的数据模型相符合的逻辑结构。

设计逻辑结构两步进行：

- (1) 将概念结构转换为关系模型
- (2) 对关系数据模型进行优化

转换时遵循下列原则：

一个实型转换为一个关系模式

一个联系转换为一个关系模式。3 种情况：

1:1 关系:

系(系编号, 系名, 电话, 地址, 系主任名, 任职年月)

系主任(系主任名, 年龄, 性别, 职称)

1:n 关系:

系(系编号, 系名, 电话, 地址, 系主任)

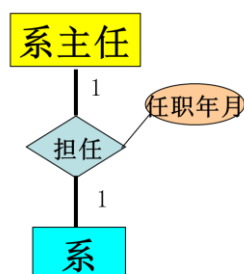
学生(学号, 系号, 姓名, 年龄, 性别)

m:n 关系:

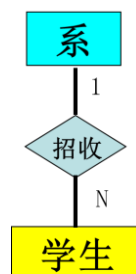
C(C#, CN, CREDIT)

S(S#, SN, AGE, SEX)

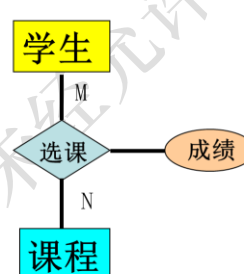
SC(S#, C#, G)



图(a) 1:1 关系



图(b) 1:n 关系



图(c) m:n 关系

图 16 关系模式转换

数据模型的优化：确定数据依赖、消除冗余联系、确定所属范式

常用分解方法：水平分解，垂直分解

7.4 物理结构与 DB 的运行

设计物理数据库结构：对要运行的事务进行详细分析，充分了解所用 RDBMS 的内部特征

关系数据库物理设计的内容：关系模式选择存取方法，设计关系、索引等物理存储结构

DBMS 常用存取方法：

索引方法：主要是 B+树索引，索引数过多会带来较多的额外开销

聚簇方法：提高按聚簇码进行查询的效率，节省存储空间，建立与维护聚簇的开销相当大

HASH 方法

易变部分与稳定部分分开存放

存取频率较高部分与存取频率较低部分，分开存放

8 数据库编程

数据失真因素：数据进入系统时的错误、系统故障、相互干扰、恶意破坏

8.1 数据库设计概述

扩展 SQL 语言的功能：

- ☐ 引入新的 SQL 子句
- ☐ 引入新的内置函数
- ☐ 引入 PL/SQL 与存储过程/存储函数

SQL 标准引入了 WITH RECURSIVE 子句，可执行递归查询。

通过高级语言实现复杂应用：

- ❖ 通过动态链接库调用的方式
- ❖ 基于嵌入式 SQL 的方式
- ❖ 基于 ODBC/JDBC 的中间件方式

8.2 过程化 SQL

SQL 的扩展，增加了过程化语句功能

基本结构是块：块之间可以互相嵌套，每个块完成一个逻辑操作

8.2.1 过程化 SQL 结构

过程化 SQL 块的基本结构：

1. 定义部分

DECLARE 变量、常量、游标、异常.....

定义的变量、常量等只能在该基本块中使用，当基本块执行结束时，定义就不再存在

2. 执行部分

BEGIN

SQL 语句、过程化 SQL 的流程控制语句

EXCEPTION

异常处理部分

END;

变量和常量的定义:

1. 变量定义

变量名 数据类型 [[NOT NULL]:=初值表达式]

变量名 数据类型 [[NOT NULL] 初值表达式]

2. 常量定义

常量名 数据类型 CONSTANT :=常量表达式

常量必须要赋予一个值，并且该值在存在期间或常量的作用域内不能改变。如果试图修改它，过程化 SQL 将返回一个异常

3. 赋值语句

变量名称 :=表达式

过程化 SQL 功能（END 结尾）:

❖ 条件控制语句：IF-THEN，IF-THEN-ELSE 和嵌套的 IF 语句

IF condition THEN

Sequence_of_statements1;

ELSE

Sequence_of_statements2;

END IF;

❖ 循环控制语句：LOOP，WHILE-LOOP 和 FOR-LOOP

(1) LOOP

Sequence_of_statements;

END LOOP;

(2) WHILE condition LOOP

Sequence_of_statements;

END LOOP;

(3) FOR count IN [REVERSE] bound1 ... bound2 LOOP

Sequence_of_statements;

END LOOP;

❖ 错误处理:

8.2.2 游标

数据库中一个指向查询结果集的指针，允许逐行读取、处理查询结果。一个游标与一个 SQL 语句相关联。

“集合操作用 SQL，逐行操作用游标”

表 5 游标的部分使用场景

场景	原因
复杂的逐行逻辑	每一行要计算、判断或触发动作
对结果集每行执行多个步骤	比如逐行插入、更新、记录日志
无法用 JOIN、聚合表达的逻辑	需要逐行“判断-操作”的控制流

游标的用户接口：

✧ 声明游标

DECLARE 游标名 [(参数 1 数据类型, 参数 2 数据类型, ...)]

CURSOR FOR

SELECT 语句;

说明性语句，这时关系 DBMS 并不执行 SELECT 语句。

✧ 打开游标

OPEN 游标名[(参数 1 数据类型, 参数 2 数据类型, ...)];

实际上执行相应的 SELECT 语句，把查询结果取到缓冲区中。这时游标处于活动状态，指针指向查询结果集中的第一条记录。

✧ 使用游标

FETCH 游标名 **INTO** 变量 1[, 变量 2,...];

把游标指针向前推进一条记录，同时将缓冲区中的当前记录取出来送至变量供过程化 SQL 进一步处理。

循环执行 FETCH 语句，逐条取出结果集中的行进行处理

✧ 关闭游标

CLOSE 游标名;

游标被关闭后就不再和原来的查询结果集相联系。被关闭的游标可以再次被打开

8.2.3 存储过程

存储过程的用户接口：

✧ 创建存储过程：CREATE OR REPLACE PROCEDURE 过程名

✧ 执行存储过程：CALL/PERFORM [PROCEDURE] 过程名([参数 1,...])

✧ 修改存储过程：

重命名：ALTER PROCEDURE 过程名 1 RENAME TO 过程名 2;

重新编译：ALTER PROCEDURE 过程名 COMPILE;

✧ 删除存储过程：DROP PROCEDURE 过程名;

存储函数：函数必须指定返回的类型

8.3 JDBC 编程

是面向 Java 语言的软件开发工具包中有关数据库的一个组成部分，提供了一组访问数据库的**应用程序编程接口**：

- ☆ JDBC 接口提供的功能必须完备
- ☆ 使用 JDBC 提供的接口而[不是 RDBMS 的接口](#)来进行应用开发

JDBC 约束力：

- ❖ 规范应用开发 (RDBMS 必须实现所有 JDBC 接口)
- ❖ 规范关系数据库管理系统应用接口

JDBC 应用系统的体系结构：

- ☆ 用户应用程序
- ☆ JDBC 驱动程序管理器
- ☆ 数据源

JDBC 操纵数据库的工作流程：

1. 加载驱动程序
2. 建立与数据库的连接
 定义连接的 URL
 利用生成的 URL 建立与数据库的连接
3. 执行 SQL 语句
 创建语句执行类对象
 执行 SQL 语句，可以通过 executeQuery、executeUpdate、execute 三种方式执行。
4. 处理结果集
5. 释放资源

9 关系数据库存储管理

9.1 数据组织

以最优的形式在外存上组织、存放庞大数据集

存储效率高，节省存储空间

存取效率高，访问速度快，代价小

9.1.1 逻辑组织方式与物理组织方式

逻辑组织方式：表空间——段——分区——数据块

- ❖ 表空间：磁盘上的一个或多个物理文件，一个物理文件只能属于一个表空间。
- ❖ 段：一个表空间逻辑上由多个段组成，每个段可以逻辑上组织不同类型的数据。
- ❖ 分区：一个段逻辑上由多个分区组成。
- ❖ 数据块：每个分区由一组连续的数据块组成（数据库的磁盘存取单元）

物理组织方式：文件——块——记录

- ❖ 文件：操作系统文件
- ❖ 块：每个文件物理上分成定长的存储单元，即操作系统的物理块。（存储分配和 I/O 的基本单位）
- ❖ 记录：物理块中存放的多条元组（记录）

9.1.2 记录表示

定长记录存储：

关系表中的每条元组占据相同大小的空间

变长字段以定长形式存储，预留最大长度空间

优势：

快速定位到记录及其属性的物理位置

增删改比较方便快捷

劣势：

浪费存储空间

变长记录存储:

存储要求: 能快速访问一条记录, 能快速访问记录中所有属性 (定长个变长属性)

存放的 3 种方式:

1. 在每条记录的头部记录该条记录的长度, 记录的每个变长字段前记录该字段的长度



图 17 变长记录存储方式 1

2. 先存放定长字段, 再存放变长字段



图 18 变长记录存储方式 2

3. 将定长字段与变长字段分开存储在不同的块中

9.1.3 块的组织

定长记录存储的块组织

表中元组依次存放在块中, **首部+记录+空闲空间**

- 增: 在空闲空间直接插入新元组
- 改: 直接在原位置修改
- 删: 回收空间, 将空闲空间加入空闲空间链表

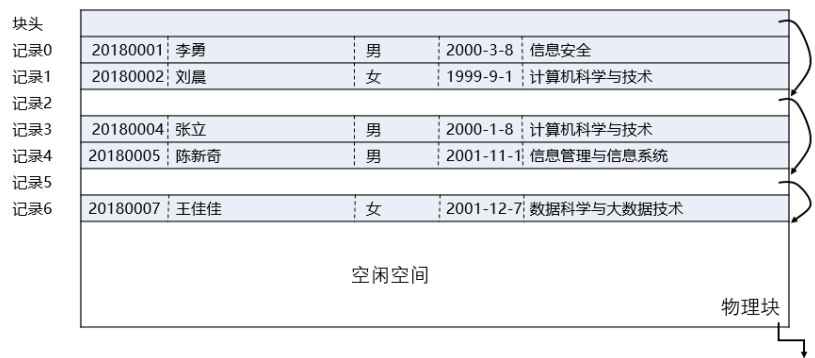


图 18 定长记录存储的块组织

变长记录存储的块组织

表中元组从块的尾部连续存放，**首部+空闲空间+记录**

- 增：从空闲空间尾部分配空间，在偏移量表中记录该元组的起始位置，调整空闲空间尾指针
- 删：在偏移量表中为该元组指针置删除标记，释放元组空间，移动物理位置在其前面的元组（保证空间连续），修改指针（被移动元组在偏移量表中的指针，空闲空间尾指针）
- 改：在原位置修改。若修改后记录在原位置放不下，会带来记录的迁移

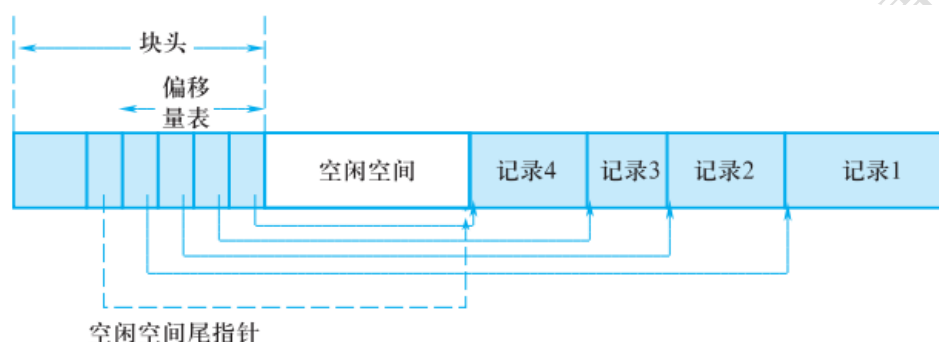


图 19 变长记录存储的块组织

9.1.4 关系表的组织

堆存储：

表中的一条记录可以存放在该表的任何块中，没有顺序要求
如果没有足够的空闲空间，就为该表**申请新的块**

顺序存储：

一个表中的各条记录根据**指定的属性或属性组的取值大小顺序**的存放
同一个表的不同块中通过**指针链接**实现有序

优势：可以高效地处理按排序属性（组）进行查询的请求

劣势：维护代价较高（在增改时需要移动已有的记录以保序）

多表聚簇存储：

不同表的元组聚簇存放在同一组块中

优势：减少连接操作带来的开销

劣势：降低某些查询的查询效率（同一表中的元组会分散在更多块中），
在更新操作时会带来更频繁的数据迁移

块头	20180001	李勇	男	2000-3-8	信息安全
记录0	20180001	81001	85	20192	81001-01
记录1	20180001	81002	96	20201	81002-01
记录2	20180001	81003	87	20202	81003-01
记录3	20180002	刘晨	女	1999-9-1	计算机科学与技术
记录4	20180002	81001	80	20192	81001-02
记录5	20180002	81002	98	20201	81002-01
记录6	20180002	81003	71	20202	81003-02
.....	20180003	王敏	女	2001-8-1	计算机科学与技术
	20180003	81001	81	20192	81001-01
	20180003	81002	76	20201	81002-02
	20180004	张立	男	2000-1-8	计算机科学与技术
	20180004	81001	56	20192	81001-02
	20180004	81002	97	20201	81002-02
	20180005	陈新奇	男	2001-11-1	信息管理与信息系统
	20180205	81003	68	20202	81003-01
	20180006	赵明	男	2000-6-12	数据科学与大数据技术
	20180007	王佳佳	女	2001-12-7	数据科学与大数据技术
	空闲空间				物理块

图 20 多表聚簇存储

B+树存储:

B+树索引的方式确定记录存放在哪个数据块中

哈希存储:

用哈希函数计算表中指定属性的哈希值，以此确定相应记录放在哪个块中

将记录的存储位置与其指定属性的取值之间建立一个对应关系，查找记录时可快速定位这条记录

9.2 索引结构

索引的优劣:**优势:**

- 表的索引块数量通常比数据块数量少得多
- 可以有高效的方法快速查找索引块
- 若索引文件足够小，可长期驻留内存缓冲区，加少 I/O 操作

劣势:

- 索引会带来额外的开销、存储索引开销、建立索引开销、维护索引开销

9.2.1 顺序表索引

也称作主索引或**聚簇索引**

稠密索引

索引块中存放每条记录的索引属性值以及指向相应记录的指针

有序索引，利用**二分查找法**在稠密索引查找指定索引项

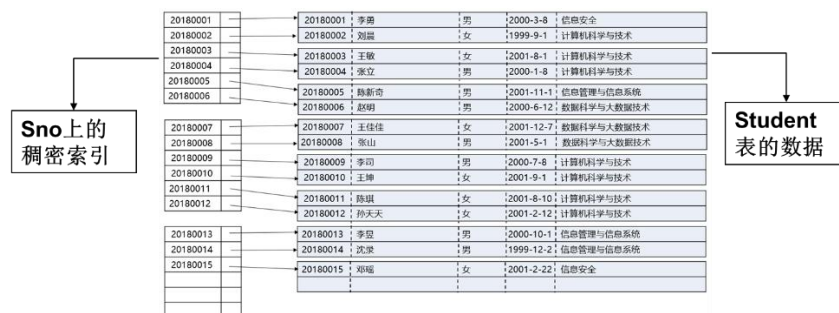


图 21 稠密索引

如果用户想在 Student 表中查找学号为 20180012 的学生信息，直接查找，不借助索引，6 次 I/O，利用稠密索引，3 次 I/O

◇ 稀疏索引

基本表的每个物理存储块只对应一个索引项，每个索引项存放每个物理块的第一条记录的索引属性值及指向该物理块的指针

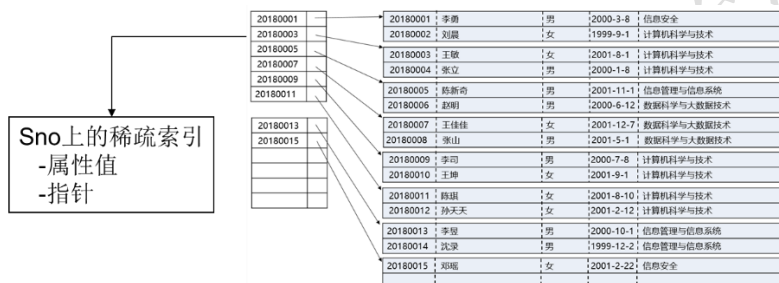


图 22 稀疏索引

在 Student 表中查找学号为 20180012 的学生信息，需要 2 次 I/O：读入索引块，读入数据块

稀疏索引的优势：

- ◇ 尺寸小于稠密索引
- ◇ 索引维护代价较小：对基本表进行增删改时，只要不是存储块第一条记录的索引属性，稀疏索引就不需要维护。

◇ 多级索引

第一级索引是稠密或稀疏索引

第二级及以上为建立在上一级索引上的稀疏索引

重复直到尺寸合适

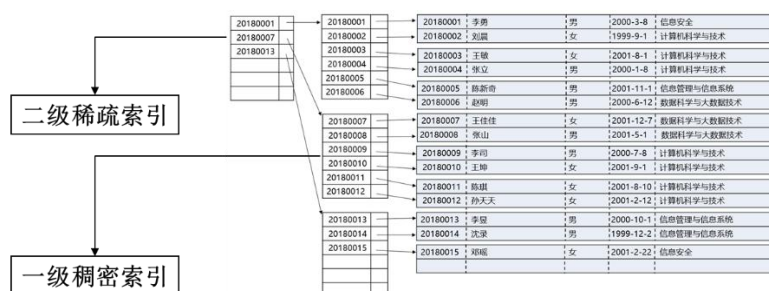


图 23 多级索引

9.2.2 辅助索引

建立在表的非排序属性上的索引

一个表最多只能建立一个主索引，但可以在不同属性上建立多个辅助索引，但只能按一个（组）属性排序

辅助索引必须是稠密索引（无序属性）

辅助索引的属性往往会取重复值，去掉可减小索引大小与查找开销，会引入指针桶去除重复索引项

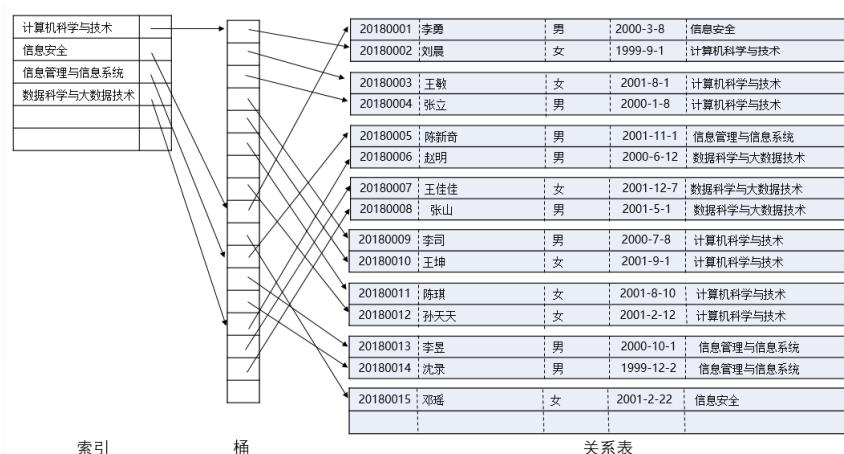


图 24 引入桶的辅助索引

9.2.3 B+树索引

一个多级索引

将索引块组织成一棵平衡树，从树根到树叶的所有路径一样长

B+树的秩（order）：一个索引块最多能存放的指针的个数

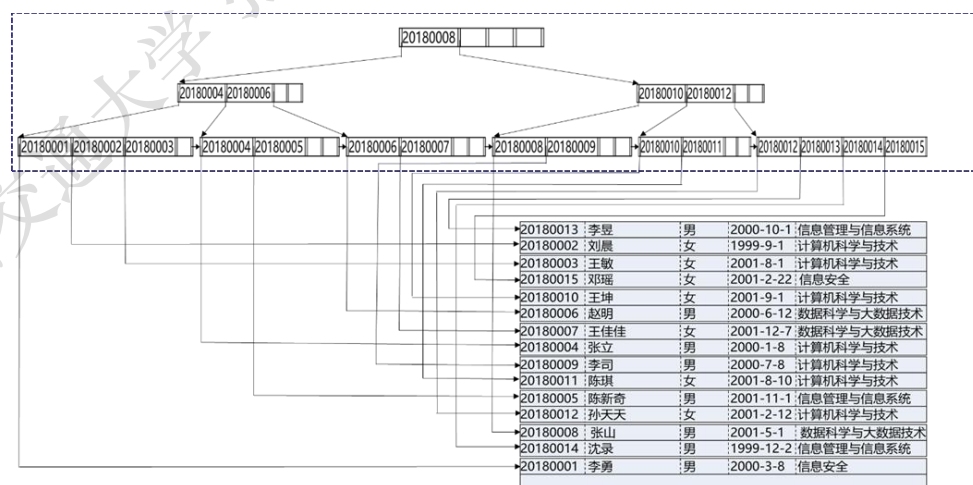


图 25 B+树索引

Student 表 Sno 属性上的 B+树索引（ $n=5$ ），一个物理块最多存放 4 个 Sno 属性值和 5 个指针，最少存放 2 个 Sno 属性值和 3 个指针，根结点除外。

B+树索引的查询：高效地完成随机查找和范围查询

- ◇ 随机查找：按照索引属性的取值进行查找
- ◇ 范围查找（between ... and ...）：用随机查找的方法分别找到范围条件的入口点和结束点，对入口点和结束点之间的属性值进行顺序搜索

维护 B+树索引：插入元组、删除元组

关键是如何维持 B+树的平衡特性

9.2.4 哈希索引

哈希表：B 个哈希桶

哈希函数：

- ◇ 将记录的索引属性值影射到哈希桶
- ◇ 每个桶存放一条或多条哈希值相同的索引项
- ◇ 每个索引项包括属性值和指向相应记录的指针

静态哈希索引：

- ◇ 哈希函数
- ◇ 哈希表：

由一组桶组成，一个桶对应一个或多个物理块

桶中存放被映射到该桶的哈希索引项

每条索引项包括索引属性值和指向相应记录的指针

注意“桶溢出”

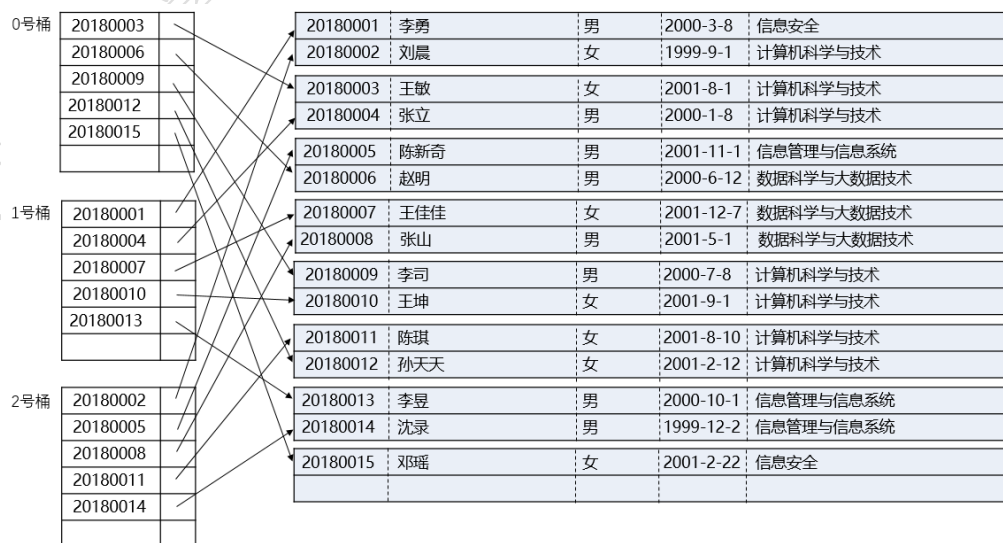


图 25 静态哈希索引

◇ 哈希索引的查找及维护

擅长做**等值查找**

插入元组时，需向哈希索引中插入相应的索引项

动态哈希索引：

随着关系表的增大，逐渐扩大桶的数目

9.2.5 位图索引

长度为 n 的位向量集合

◇ n ：索引属性的基数（可能取值的个数）

◇ 每一个位向量对应于索引属性的一个可能的取值

能够有效处理多值查询（**建立多个位图索引**）

Ssno	Sname	Ssex	Sbirthdate	Smajor	男	女
20180001	李勇	男	2000-3-8	信息安全	1	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1
20180004	张立	男	2000-1-8	计算机科学与技术	1	0
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	1	0
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	0	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	1	0
20180009	李司	男	2000-7-8	计算机科学与技术	1	0
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1
20180013	李昱	男	2000-10-1	信息管理与信息系统	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	1	0
20180015	邓瑶	女	2001-2-22	信息安全	0	1

10对应男生
01对应女生

图 26 位图索引

优势：使用位操作来快速回答用户查询或快速定位满足条件的元组集合，减少对基本表的全表扫描，提高查询效率

劣势：位图索引的大小与列的基数成正比，基数大的列其位图索引会非常庞大，因此它只适用于基数小的属性列

编码位图索引

改进标准位图索引：通过对属性值编码，减少索引位向量的个数，从而能够应对有较高基数的列

编码位图索引的优劣

优势：减少索引位个数

劣势：查询时需要访问所有位向量才能完成

10 查询处理和查询优化

10.1 关系数据库系统的查询处理与优化

RDBMS 查询处理阶段：查询分析，查询检查，查询优化，查询执行

查询处理选择操作的实现：

简单的全表扫描方法、索引（或散列）扫描方法

查询处理连接操作的实现：

- ✧ 嵌套循环方法(nested loop)
- ✧ 排序-合并方法(sort-merge join 或 merge join)
- ✧ 索引连接(index join)方法
- ✧ Hash Join 方法

查询优化的总目标：

选择有效的策略，求得给定关系表达式的值，使得查询代价最小

10.2 代数优化

SQL 的扩展，增加了过程化语句功能

基本结构是块：块之间可以互相嵌套，每个块完成一个逻辑操作

10.2.1 关系代数表达式等价变换规则

两个关系表达式 E_1 和 E_2 是等价的，可记为 $E_1 \equiv E_2$

常用的等价变换规则：

❖ 连接、笛卡尔积交换律

设 E_1 和 E_2 是关系代数表达式， F 是连接运算的条件，则有

$$E_1 \times E_2 \equiv E_2 \times E_1 \quad E_1 \bowtie E_2 \equiv E_2 \bowtie E_1 \quad E_1 \bowtie_F E_2 \equiv E_2 \bowtie_F E_1$$

❖ 连接、笛卡尔积的结合律

设 E_1, E_2, E_3 是关系代数表达式， F_1 和 F_2 是连接运算的条件，则有

$$(E_1 \times E_2) \times E_3 \equiv E_1 \times (E_2 \times E_3)$$

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \bowtie_F E_2) \bowtie_{F'} E_3 \equiv E_1 \bowtie_{F'} (E_2 \bowtie_F E_3)$$

- ❖ 投影的串接定律
- ❖ 选择的串接定律

$\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_1 \wedge F_2}(E)$, E 是关系代数表达式, F_1 、 F_2 是选择条件。

- ❖ 选择与投影操作的交换律
- ❖ 选择与笛卡尔积的交换律
- ❖ (非重点, 省略)

10.2.2 查询树的启发式优化

以 SELECT 语句对应投影操作 (π)

以 FROM 语句对应笛卡尔乘积 ($\times$)

以 WHERE 语句对应选择操作 (δ)

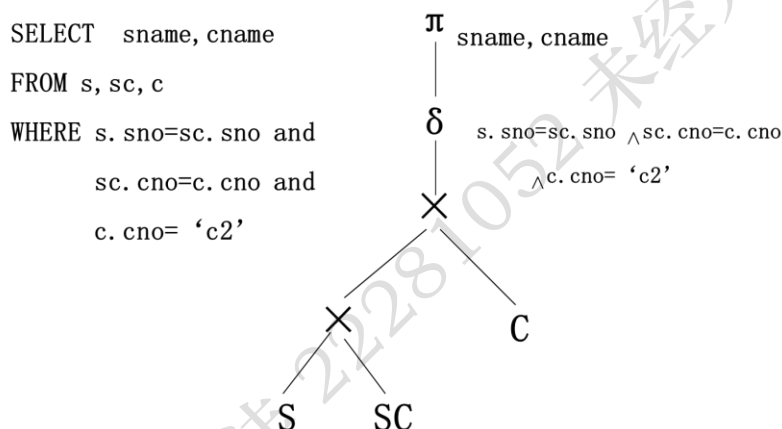


图 27 查询树案例

启发式规则:

- ✧ 选择运算应尽可能先做。在优化策略中这是最重要、最基本的一条
- ✧ 把投影运算和选择运算同时进行

如有若干投影和选择运算, 并且它们都对同一个关系操作, 则可以在扫描此关系的同时完成所有的这些运算以避免重复扫描关系

- ✧ 把投影同其前或其后的双目运算结合起来
- ✧ 把某些选择同在它前面要执行的笛卡尔积结合起来成为一个连接运算
- ✧ 找出公共子表达式

如果这种重复出现的子表达式的结果不是很大的关系并且从外存中读入这个关系比计算该子表达式的时间少得多, 则先计算一次公共子表达式并把结果写入中间文件是合算的

当查询的是视图时, 定义视图的表达式就是公共子表达式的情况

案例：检索至少学习 LIU 老师所授一门课的女同学的学号和姓名，用 SQL 表达为：

```
SELECT s#,sn
FROM s,sc,c
WHERE t='LIU' and sex='女' and
      s.s#=sc.s# and sc.c#=c.c#
```

优化过程如下：

1. 画出原始的查询树

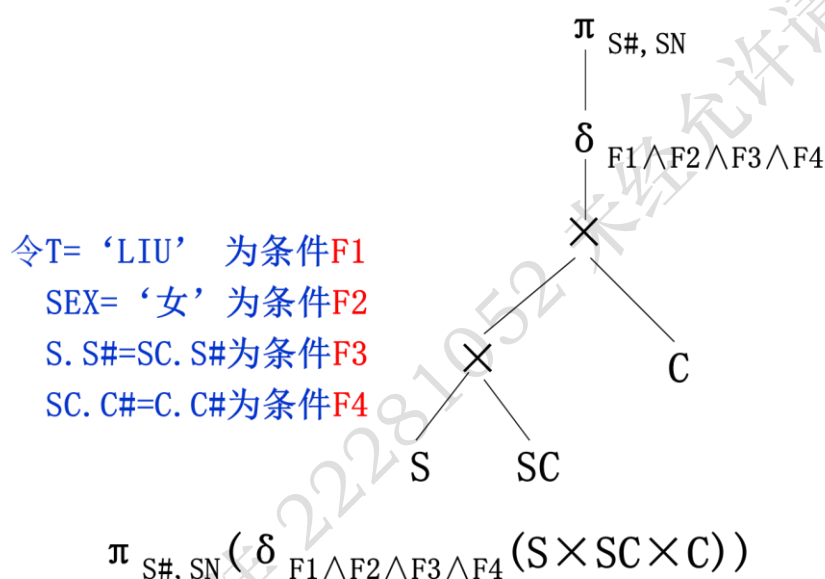


图 28 查询树的启发式优化步骤 1

2. 将选择条件移向叶子端点

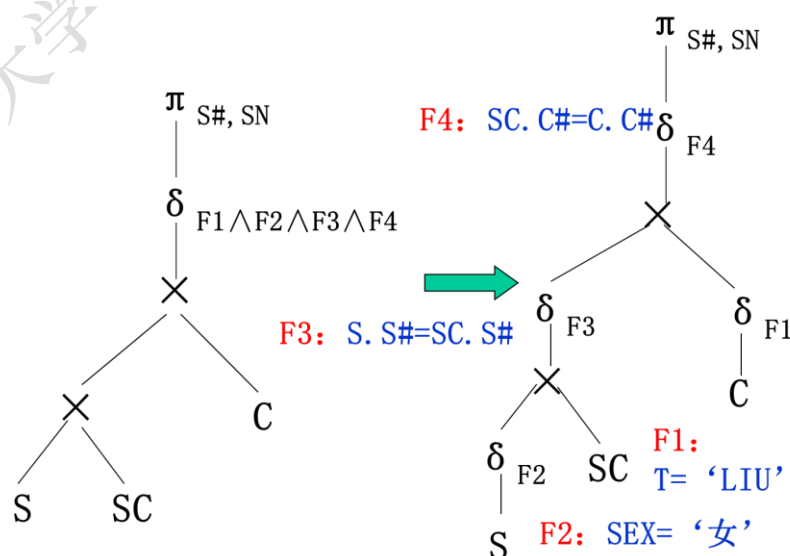


图 29 查询树的启发式优化步骤 2

3. 在联接前先做投影操作将无关的属性去掉、

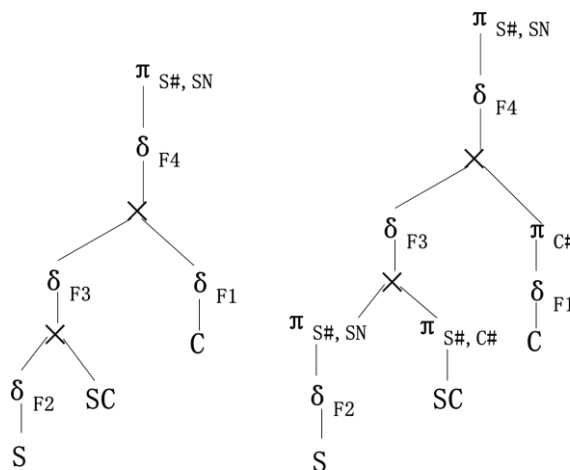


图 29 查询树的启发式优化步骤 3

4. 用联接操作代替笛卡尔积

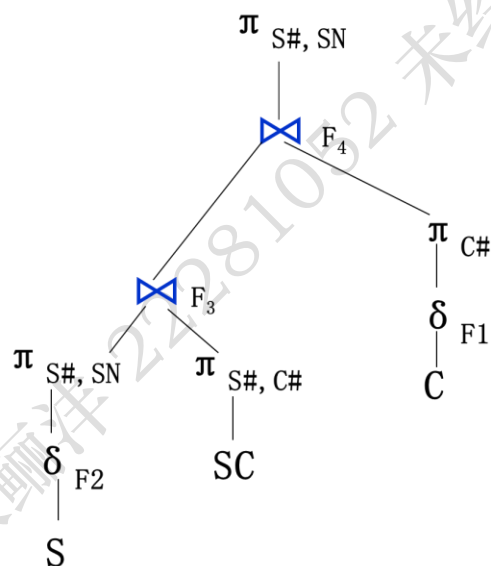


图 30 查询树的启发式优化步骤 4

5. 分组

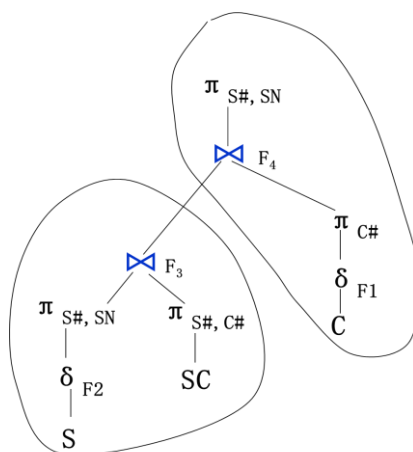


图 31 查询树的启发式优化步骤 5

10.3 物理优化

代数优化改变查询语句中操作的次序和组合，不涉及底层的存取路径

10.3.1 基于启发式规则的存取路径选择优化

选择操作的启发式规则：

1. 对于小关系，使用全表顺序扫描，即使选择列上有索引

对于大关系，启发式规则有：

2. 对于选择条件是主码=值的查询，查询结果最多是一个元组，可以选择主码索引
3. 对于选择条件是非主属性=值的查询，并且选择列上有索引
4. 对于选择条件是属性上的非等值查询或者范围查询，并且选择列上有索引
5. 对于用 AND 连接的合取选择条件，如果有涉及这些属性的组合索引，优先采用组合索引扫描方法
6. 对于用 OR 连接的析取选择条件，一般使用全表顺序扫描

连接操作的启发式规则：

1. 如果 2 个表都已经按照连接属性排序：选用排序-合并方法
2. 如果一个表在连接属性上有索引：选用索引连接方法
3. 如果上面 2 个规则都不适用，其中一个表较小：选用 Hash join 方法
4. 可以选用嵌套循环方法，并选择其中较小的表
- 5.

10.3.2 基于代价的优化

10.3.2.1 统计信息

数据字典中存储优化器需要的统计信息：

❖ 对每个基本表

- 该表的元组总数(N)
- 元组长度(l)
- 占用的块数(B)
- 占用的溢出块数(BO)

❖ 对基表的每个列

- 该列不同值的个数(m)
- 选择率(f)
- 如果不同值的分布是均匀的, $f=1/m$
- 如果不同值的分布不均匀, 则每个值的选择率=具有该值的元组数/ N
- 该列最大值
- 该列最小值
- 该列上是否已经建立了索引
- 索引类型(B+树索引、Hash 索引、聚集索引)

❖ 对索引(如 B+树索引)

- 索引的层数(L)
- 不同索引值的个数
- 索引的选择基数 S (有 S 个元组具有某个索引值)
- 索引的叶结点数(Y)

10.3.2.2 代价估算

1. 全表扫描 (Full Table Scan)

适用场景: 没有索引或查询范围较大时。

代价估算:

- ❖ 整表扫描: $Cost = B$
- ❖ 主码=值时, 平均搜索代价: $Cost = B / 2$

说明: B 表示基本表的块数。

2. 索引扫描 (Index Scan)

适用场景: 查询条件使用索引字段, 如主键或建立了索引的列。

代价估算:

- ❖ 主码=值 (使用 B+树主索引): $Cost = L + 1$
- ❖ 非主码字段=值 (使用辅助索引, 满足 S 条记录): $Cost = L + S$

说明:

L 为 B+树高度

S 为满足条件的记录数。

3. 嵌套循环连接 (Nested Loop Join)

适用场景：两表连接，外表小或无索引。

代价估算：

❖ 不写连接结果： $Cost = B_r + \left(\frac{B_s}{K-1}\right) \cdot B_r$

❖ 写连接结果： $Cost = B_r + \left(\frac{B_s}{K-1}\right) \cdot B_r + \frac{F_{rs} \cdot N_r \cdot N_s}{M_{rs}}$

☆ B_r 、 B_s 为两表块数

☆ F_{rs} 为连接选择性

☆ N_r 、 N_s 为行数

☆ M_{rs} 为块因子。

4. 排序-合并连接 (Sort-Merge Join)

适用场景：连接字段有序或可以排序。

代价估算：

若已排序： $Cost = B_r + B_s + \frac{F_{rs} \cdot N_r \cdot N_s}{M_{rs}}$

若未排序：需先排序再连接，每个文件排序成本大致为：

$$SortCost = 2B + 2B \cdot \log_2 B$$

总成本： $Cost = Sort(R) + Sort(S) + B_r + B_s + \frac{F_{rs} \cdot N_r \cdot N_s}{M_{rs}}$

表 7 算法对比总结表

算法类型	代价估算公式 (简化)	关键词
全表扫描	B 或 $B/2$	从头扫，平均翻半表
索引扫描 (主码)	$L + 1$	B+树高度加一
索引扫描 (非码)	$L + S$	多个匹配要多找
嵌套循环连接	$B_r + \left(\frac{B_s}{K-1}\right) \cdot B_r$	外扫一次，内重复
嵌套连接+写结果	$+ \frac{F_{rs} \cdot N_r \cdot N_s}{M_{rs}}$	加写入代价
排序-合并连接 (已排序)	$B_r + B_s + \text{连接结果块数}$	有序快
排序-合并连接 (未排序)	排序代价 + 上述	无序要先排

11 数据库恢复技术

11.1 事务

事务是用户定义的一个数据库操作序列。这些操作要么都做，要么都不做，是一个不可分割的工作单位。是恢复和并发控制的基本单位

在关系数据库中，一个事务可以是一条或多条 SQL 语句,也可以包含一个或多个程序。

一个程序通常包含多个事务

事务的 ACID 性质:

- ☆ 原子性：一个事务中所有对数据库的操作是一个不可分割的操作序列
- ☆ 一致性：数据不会因事务的执行而遭受破坏
- ☆ 隔离性：在并发事务被执行时，系统应保证与这些事务先后单独执行时结果一样，使事务如同在单用户环境下执行一样
- ☆ 永久性：要求对数据库的全部操作提交后，事务对数据库的所有更新应永久地反映在数据库中

11.2 数据库故障

1. 事务故障

- ☆ 可以预期的事务的故障

可以预期的事务的故障（可以发现的事务故障），并且可以让事务回退

- ☆ 不可预期的事务的故障

事务内部更多的故障是非预期的，是不能由应用程序处理的（运算溢出、并发事务发生死锁而被选中撤销该事务、违反某些完整性等）

事务故障的恢复：撤消事务（UNDO）

事务故障意味着事务没有到达预期的终点。因此，数据库可能处于不正确状态。系统就要强行回滚此事务，即撤消该事务已经对数据库所做的任何修改，使得该事务好像根本没有启动一样。

2. 系统故障

称为软故障，是指造成系统停止运转的任何事件，使得系统要重新启动。（特定类型的硬件错误如 CPU 故障、OS 故障、DBMS 代码错误、系统断电）

发生系统故障时，事务未提交

恢复策略：强行撤消 **(UNDO) 所有未完成事务**

发生系统故障时，事务已提交

恢复策略：**重做 (REDO) 所有已提交的事务**

3. 介质故障

外存故障：磁盘损坏、磁头碰撞、操作系统的某种潜在错误等

4. 计算机病毒

11.3 数据库恢复技术

恢复操作的基本原理：冗余

利用存储在系统其它地方的冗余数据来重建数据库中已被破坏或不正确的那部分数据

如何建立冗余数据：

1. 数据转储

表 7 数据转储分类

		转储状态	
		动态转储	静态转储
转储方式	海量转储	动态海量转储	静态海量转储
	增量转储	动态增量转储	静态增量转储

2. 登录日志文件

日志文件 (log) 是用来记录事务对数据库的更新操作的文件

日志文件的格式

以记录为单位的日志文件

- ☆ 各个事务的开始标记(BEGIN TRANSACTION)
- ☆ 各个事务的结束标记(COMMIT 或 ROLLBACK)
- ☆ 各个事务的所有更新操作

以数据块为单位的日志文件

☆ 事务标识（标明是哪个事务）

☆ 被更新的数据块

日志文件的作用：

进行事务故障恢复

进行系统故障恢复

协助后备副本进行介质故障恢复

登记的次序严格按并行事务执行的时间次序

必须先写日志文件，后写数据库

11.4 恢复策略

事务故障的恢复：

事务在运行至正常终止点前被终止

由恢复子系统利用日志文件撤消（UNDO）此事务已对数据库进行的修改

系统故障的恢复：

系统故障造成数据库不一致状态的原因：

未完成事务对数据库的更新已写入数据库

已提交事务对数据库的更新还留在缓冲区没来得及写入数据库

恢复方法：

◇ Undo 故障发生时未完成的事务

◇ Redo 已完成的事务

介质故障的恢复：

重装数据库

重做已完成的事务

11.5 具有检查点的恢复技术

检查点到故障点之间发生的事务

没有提交的 UNDO 回滚

提交了的 REDO 重做

检查点记录的内容:

建立检查点时刻所有正在执行的事务清单

这些事务最近一个日志记录的地址

重新开始文件的内容:

记录各个检查点记录在日志文件中的地址

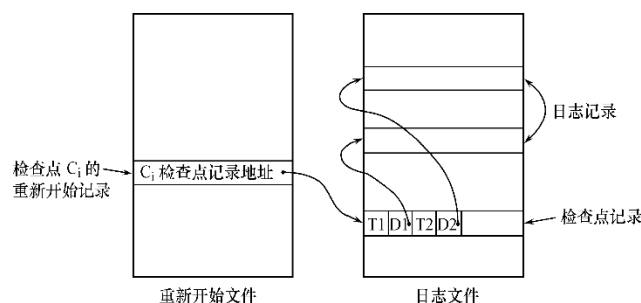


图 32 具有检查点的日志文件和重新开始文件

系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略

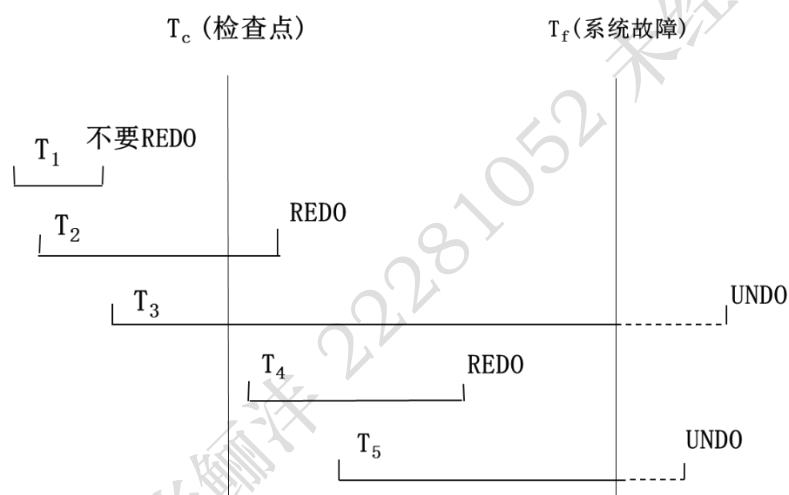


图 33 检查点时序

- ❏ $T1$: 在检查点之前提交
- ❏ $T2$: 在检查点之前开始执行，在检查点之后故障点之前提交
- ❏ $T3$: 在检查点之前开始执行，在故障点时还未完成
- ❏ $T4$: 在检查点之后开始执行，在故障点之前提交
- ❏ $T5$: 在检查点之后开始执行，在故障点时还未完成

恢复策略:

- ❏ $T3$ 和 $T5$ 在故障发生时还未完成，所以予以撤销
- ❏ $T2$ 和 $T4$ 在检查点之后才提交，它们对数据库所做的修改在故障发生时可能还在缓冲区中，尚未写入数据库，所以要 REDO
- ❏ $T1$ 在检查点之前已提交，所以不必执行 REDO 操作

11.6 数据库镜像

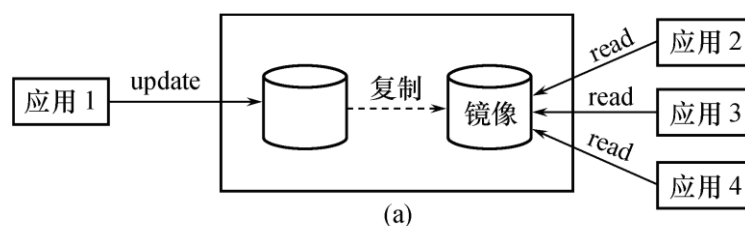


图 34 数据库镜像

出现介质故障时

- 可由镜像磁盘继续提供使用
- 同时 DBMS 自动利用镜像磁盘数据进行数据库的恢复
- 不需要关闭系统和重装数据库副本

没有出现故障时

- 可用于并发操作
- 一个用户对数据加排他锁修改数据，其他用户可以读镜像数据库上的数据，而不必等待该用户释放锁

12 并发控制

12.1 多事务执行问题

不同的多事务执行方式：事务串行执行、交叉并发方式、同时并发方式
并发控制机制的任务：

- ✧ 对并发操作进行正确调度
- ✧ 保证事务的隔离性
- ✧ 保证数据库的一致性

并发操作带来的数据不一致性

❖ 丢失修改 (Lost Update)

两个事务同时修改同一数据，后提交的事务覆盖了前一个事务的修改，导致前一个修改“丢失”。

通俗比喻：两人同时编辑同一个文档，一个人保存后，另一个人再保存，前者的修改被覆盖。

❖ 不可重复读 (Non-repeatable Read)

一个事务内多次读取同一数据，每次的值可能不同，因为另一个事务修改并提交了该数据。

通俗比喻：上午查余额是 100，下午再查变成了 50，虽然你自己什么都没做。

❖ 读“脏”数据 (Dirty Read)

一个事务读取了另一个未提交事务修改的数据。如果对方回滚，这个读取就变成“脏”的。

通俗比喻：朋友告诉你一个秘密，突然反悔说“当我没说”，但你已经听见了。

❖ 幻读 (Phantom Read) (不可重复读一种)

一个事务中两次执行相同查询，第二次结果多了几行，是其他事务新增的数据。

通俗比喻：上午点名有 20 人，下午再查发现 25 人，期间你啥也没干，是别人偷偷加了人。

数据不一致性：由于并发操作破坏了事务的隔离性

并发控制的主要技术：封锁、时间戳、乐观控制法

12.2 封锁

封锁就是事务 T 在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁

基本封锁类型

❖ 排它锁（Exclusive Locks，简记为 X 锁，写锁）

若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A，其它任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁

保证其他事务在 T 释放 A 上的锁之前不能再读取和修改 A

❖ 共享锁（Share Locks，简记为 S 锁，读锁）

若事务 T 对数据对象 A 加上 S 锁，则其它事务只能再对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁

保证其他事务可以读 A，但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改

表 8 锁的相容矩阵

$\begin{matrix} T_2 \\ T_1 \end{matrix}$	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

Y=Yes，相容的请求 N=No，不相容的请求

最左边一列表示事务 T1 已经获得的数据对象上的锁的类型，其中横线表示没有加锁。

最上面一行表示另一事务 T2 对同一数据对象发出的封锁请求。

解决丢失修改问题：

T_1	T_2
① Xlock A	没有丢失修改
② R(A)=16	
③ $A \leftarrow A-1$	Xlock A
W(A)=15	等待
Commit	等待
Unlock A	等待
④	获得Xlock A
	R(A)=15
	$A \leftarrow A-1$
⑤	W(A)=14
	Commit
	Unlock A

图 36 解决丢失修改问题

解决不可重复读问题:

T ₁	T ₂
① Slock A Slock B R(A)=50 R(B)=100 求和=150 ②	
③ R(A)=50 R(B)=100 求和=150 Commit Unlock A Unlock B ④	Xlock B 等待 等待 等待 等待 等待 等待 获得XlockB R(B)=100 B←B*2 W(B)=200 Commit Unlock B

可重复读

- 事务T1在读A, B之前, 先对A, B加S锁
- 其他事务只能再对A, B加S锁, 而不能加X锁, 即其他事务只能读A, B, 而不能修改
- 当T2为修改B而申请对B的X锁时被拒绝只能等待T1释放B上的锁
- T1为验算再读A, B, 这时读出的B仍是100, 求和结果仍为150, 即可重复读
- T1结束才释放A, B上的S锁。T2才获得对B的X锁

图 37 解决不可重复读问题

解决读“脏”数据问题:

T ₁	T ₂
① Xlock C R(C)=100 C←C*2 W(C)=200 ②	
③ ROLLBACK (C恢复为100) Unlock C ④	Slock C 等待 等待 等待 等待 获得Slock C R(C)=100
⑤	Commit C Unlock C

不读“脏”数据

- 事务T1在对C进行修改之前, 先对C加X锁, 修改其值后写回磁盘
- T2请求在C上加S锁, 因T1已在C上加了X锁, T2只能等待
- T1因某种原因被撤销, C恢复为原值100
- T1释放C上的X锁后T2获得C上的S锁, 读C=100。避免了T2读“脏”数据

图 38 解决读“脏”数据问题

封锁协议:

- 一级封锁协议, 事务 T 在修改数据 R 之前必须对其加 X 锁, 直到事务结束才释放。一级封锁协议可防止丢失修改, 但不能保证可重复读和不读“脏”数据。
- 二级封锁协议, 在一级封锁协议的基础上增加事务 T 在读取数据 R 之前必须先对其加 S 锁, 读完就可释放 S 锁。二级封锁协议因为读完数据就释放 S 锁, 不能保证可重复读。
- 三级封锁协议, 在一级封锁协议的基础上增加事务 T 在读取数据 R 之前必须先对其加 S 锁, 直到事务结束才释放。

四级隔离级别：

❖ 读未提交 (read uncommitted)

对应一级封锁协议，提供最大的事务并发度，仅能避免丢失修改

❖ 读已提交 (read committed)

对应二级封锁协议，事务并发度减弱，能避免丢失修改和脏读

❖ 可重复读 (repeatable read)

对应三级封锁协议，读取数据时加 S 锁，直到事务结束才释放。事务并发度进一步减弱，能避免丢失修改、脏读和不可重复读

❖ 可串行化 (serializable)

更强的封锁协议，提供最小的事务并发度，能避免所有并发控制问题（包括幻读）

12.3 活锁与死锁

多个事务（或线程）虽然没有阻塞，但一直在变换状态，却始终无法完成任务，即：一直“活着”，但就是干不成活儿。

避免活锁：采用先来先服务的策略

当多个事务请求封锁同一数据对象时

按请求封锁的先后次序对这些事务排队

该数据对象上的锁一旦释放，首先批准申请队列中第一个事务获得锁

死锁：多个事务（或线程）相互等待对方释放资源，结果谁也不释放，程序永久阻塞。

预防死锁的方法：一次封锁法、顺序封锁法

死锁的诊断：超时法、事务等待图法

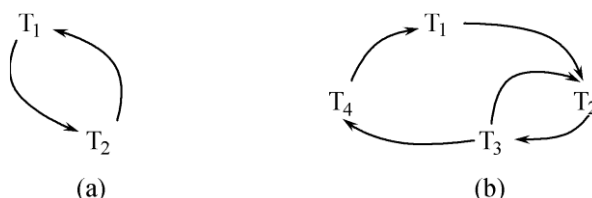


图 39 等待图

死锁是大家都在等，谁也不动；活锁是大家都在让，谁也走不了。

12.4 并发调度的可串行性

12.4.1 可串行化调度

可串行化(Serializable)调度

多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同

可串行性(Serializability)

是并发事务正确调度的准则

一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度

T ₁	T ₂	T ₁	T ₂
Slock B		Slock B	
Y=R(B)=2		Y=R(B)=2	
	Slock A	Unlock B	
	X=R(A)=2	Xlock A	
Unlock B			Slock A
	Unlock A	A=Y+1=3	等待
Xlock A		W(A)	等待
A=Y+1=3		Unlock A	等待
W(A)			X=R(A)=3
	Xlock B		Unlock A
	B=X+1=3		Xlock B
	W(B)		B=X+1=4
Unlock A			W(B)
	Unlock B		Unlock B

图(a) 不可串行化调度，错误的调度

图(b) 可串行化调度，正确的调度

图 40 串行化调度

事务 T1: 读 B; A=B+1; 写回 A

事务 T2: 读 A; B=A+1; 写回 B

12.4.2 冲突可串行化调度

一个调度是冲突可串行化，一定是可串行化的调度

可串行化调度的充分条件:

一个调度 Sc 在保证冲突操作的次序不变的情况下，通过交换两个事务不冲突操作的次序得到另一个调度 Sc'，如果 Sc' 是串行的，称调度 Sc 为冲突可串行化的调度。冲突可串行化调度是可串行化调度的充分条件，不是必要条件。

冲突操作: 指不同的事务对同一个数据的读写操作和写写操作

12.5 两段锁协议 (2PL)

指所有事务必须分两个阶段对数据项加锁和解锁

- ☐ 在对任何数据进行读、写操作之前，事务首先要获得对该数据的封锁
- ☐ 在释放一个封锁之后，事务不再申请和获得任何其他封锁

“两段”锁的含义

事务分为两个阶段：

- ☐ 第一阶段是获得封锁，也称为扩展阶段

事务可以申请获得任何数据项上的任何类型的锁，但是不能释放任何锁

- ☐ 第二阶段是释放封锁，也称为收缩阶段

事务可以释放任何数据项上的任何类型的锁，但是不能再申请任何锁

在一个事务中，**加锁阶段 (Locking phase)**：只加锁不解锁；

解锁阶段 (Unlocking phase)：一旦开始释放锁，就不能再加新锁。

只要在释放锁之后还加锁，就违反两段锁协议。

12.6 封锁粒度

封锁对象的大小称为封锁粒度

封锁的对象：逻辑单元，物理单元

在 RDBMS 中，封锁对象：

- ☐ 逻辑单元：属性值、属性值集合、元组、关系、索引项、整个索引、整个数据库等
- ☐ 物理单元：页（数据页或索引页）、物理记录等

封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；

封锁的粒度越小，并发度较高，但系统开销也就越大

在多粒度封锁中一个数据对象可能以两种方式封锁：显式封锁和隐式封锁

- ☐ 显式封锁：直接加到数据对象上的封锁
- ☐ 隐式封锁：该数据对象没有独立加锁，是由于其上级结点加锁而使该数据对象加上了锁

意向锁

如果对一个结点加意向锁，则说明该结点的下层结点正在被加锁

对任一结点加基本锁，必须先对它的上层结点加意向锁

❖ 意向共享锁(Intent Share Lock, 简称 IS 锁)

如果对一个数据对象加 IS 锁，表示它的后裔结点拟（意向）加 S 锁。

❖ 意向排它锁(Intent Exclusive Lock, 简称 IX 锁)

如果对一个数据对象加 IX 锁，表示它的后裔结点拟（意向）加 X 锁。

❖ 共享意向排它锁(Share Intent Exclusive Lock, 简称 SIX 锁)

如果对一个数据对象加 SIX 锁，表示对它加 S 锁，再加 IX 锁，即 $SIX = S + IX$

$T_1 \backslash T_2$	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

Y=Yes, 表示相容的请求 N=No, 表示不相容的请求

(a) 数据锁的相容矩阵

锁的强度：指它对其他锁的排斥程度

一个事务在申请封锁时以强锁代替弱锁是安全的，反之则不然

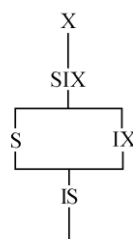


图 42 锁的强度偏序关系

具有意向锁的多粒度封锁方法：

- ☐ 申请封锁时应该按自上而下的次序进行
- ☐ 释放封锁时则应该按自下而上的次序进行

具有意向锁的多粒度封锁方法提高了系统的并发度，减少了加锁和解锁的开销